

Содержание

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	4
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ВВЕДЕНИЕ	7
1 Анализ квантовых угроз и нормативной базы	9
1.1 Квантовые вычисления как новый вектор угроз для современной криптографии	9
1.2 Алгоритм Шора: теоретическая угроза асимметричной криптографии	11
1.3 Алгоритм Гровера: влияние на симметричную криптографию	13
1.4 Угроза «перехвати сейчас – расшифруй потом» (HN DL)	14
1.5 Стандарты NIST в области постквантовой криптографии . . .	16
1.6 Рекомендации ТК 26 в контексте постквантового перехода . .	17
2 Анализ существующих аналогов	19
2.1 IBM Quantum Safe (экосистема)	20
2.2 CodeQL	22
2.3 CBOMkit	24
2.4 CryptoScan	25
2.5 SonarQube	26
2.6 CycloneDX cdxgen	28
2.7 QApp	29
2.8 Выводы по результатам анализа аналогов	30
3 Проектирование архитектуры инструмента	31
3.1 Формирование требований	31
3.1.1 Требования к входным данным и запуску инструмента	32
3.1.2 Требования к инвентаризации файлов проекта	33
3.1.3 Требования к модулю статического анализа исходного кода	33
3.1.4 Требования к базе данных криптографических сигнатур	34
3.1.5 Требования к модулю анализа сертификатов	34

3.1.6	Требования к модулю оценки рисков и формирования плана миграции	35
3.1.7	Требования к отчетности	36
3.1.8	Нефункциональные требования	36
3.2	Общая архитектура и процесс обработки	36
4	Реализация инструмента	40
4.1	Общая организация разработки	40
4.2	Реализация конфигурации и командного интерфейса	40
4.3	Реализация базы уязвимых функций	42
4.4	Реализация инвентаризации проекта	46
4.5	Реализация общего интерфейса анализаторов	49
4.6	Реализация анализатора регулярных выражений	50
4.7	Реализация AST-анализатора	51
4.7.1	Парсинг единицы трансляции	52
4.7.2	jjj	52
4.7.3	Сбор локальных значений	53
4.7.4	Сбор полей классов и побочных эффектов	54
4.7.5	Построение достижимости	55
4.7.6	Разрешение аргументов	56
4.7.7	Ограничения AST-анализатора	56
4.8	Реализация анализа сертификатов	57
4.9	Реализация оценки рисков	58
4.10	Реализация плана миграции	61
4.11	Реализация CBOM и генератора отчетов	62
5	Тестирование и оценка	66
5.1	Общий подход к тестированию	66
5.2	Тестовые данные	68
5.3	Модульные тесты	71
5.3.1	База данных уязвимых функций (<i>VulnDatabase</i>)	71
5.3.2	Инвентаризация проекта (<i>ProjectScanner</i>)	72
5.3.3	Regex-анализатор (<i>RegexAnalyzer</i>)	74
5.3.4	AST-анализатор (<i>ASTAnalyzer</i>)	75
5.3.5	Оценщик риска (<i>RiskAssessor</i>)	76

5.3.6	Модуль метрик (<i>MetricsCalculator</i>)	78
5.4	Интеграционные тесты	79
5.5	Оценка результатов тестирования	80
6	Анализ полученных результатов	81
6.1	Соответствие реализации поставленным требованиям	81
6.2	Сравнение режимов анализа	82
6.3	Оценка практической применимости	84
6.4	Ограничения реализации	85
6.5	Направления дальнейшего развития	85
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	87

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

AES	– Advanced Encryption Standard — расширенный стандарт симметричного шифрования
API	– Application Programming Interface — интерфейс прикладного программирования
AST	– Abstract Syntax Tree — абстрактное синтаксическое дерево
BFS	– Breadth-First Search — обход графа в ширину
CBOM	– Cryptography Bill of Materials — криптографическая ведомость материалов
CLI	– Command Line Interface — интерфейс командной строки
DER	– Distinguished Encoding Rules — правила однозначного кодирования ASN.1
DH/DHE	– Diffie-Hellman (Ephemeral) — (эфемерный) обмен ключами Диффи-Хеллмана
DSA	– Digital Signature Algorithm — алгоритм цифровой подписи
ECC	– Elliptic Curve Cryptography — криптография на эллиптических кривых
ECDH	– Elliptic Curve Diffie-Hellman — обмен ключами на эллиптических кривых
ECDSA	– Elliptic Curve Digital Signature Algorithm — алгоритм цифровой подписи на ЭК
FIPS	– Federal Information Processing Standard — федеральный стандарт обработки информации (США)
FN	– False Negative — ложноотрицательный результат
FP	– False Positive — ложноположительный результат
HNDL	– Harvest Now, Decrypt Later — атака «собери сейчас, расшифруй потом»
JSON	– JavaScript Object Notation — текстовый формат обмена данными
KEM	– Key Encapsulation Mechanism — механизм инкапсуляции ключей
MD5	– Message Digest Algorithm 5 — криптографическая хеш-функция

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Квантово-уязвимый алгоритм	криптографический алгоритм, безопасность которого может быть нарушена за полиномиальное время с использованием квантового компьютера достаточной мощности.
Постквантовая криптография	раздел криптографии, посвящённый разработке криптографических алгоритмов, устойчивых к атакам как классических, так и квантовых компьютеров.
Криптографический манифест (CBOM)	расширение формата CycloneDX Software Bill of Materials, описывающее криптографические активы программной системы: алгоритмы, ключи, сертификаты и протоколы.
Статический анализ	метод проверки программного кода без его выполнения, позволяющий выявлять дефекты, уязвимости и использование устаревших API.
Инкапсуляция ключей (КЕМ)	криптографический примитив, позволяющий безопасно передавать симметричный ключ с использованием асимметричной криптографии.
Криптографическая гибкость (Crypto-Agility)	свойство программной системы, позволяющее заменять криптографические алгоритмы без существенной переработки архитектуры.

ВВЕДЕНИЕ

В настоящее время стремительное развитие квантовых вычислений формирует совершенно новый класс угроз для современной криптографической инфраструктуры. В частности, алгоритм Шора, запущенный на квантовом компьютере, способен за полиномиальное время взломать любую систему с открытым ключом, что разрушает фундамент современной асимметричной криптографии: RSA, ECDSA и протокол Диффи-Хеллмана.

Согласно ежегодному докладу Global Risk Institute за 2025-2026 год, вероятность появления криптографически значимого квантового компьютера в течение ближайших десяти лет составляет от 28% до 49%, что является максимальным показателем за семилетнюю историю публикаций. По оценкам ведущих исследовательских центров, создание квантового компьютера с числом логических кубитов, достаточным для взлома RSA-2048 в реальные сроки, прогнозируется в период 2030-2040 годов.

В ответ на эту угрозу Национальный институт стандартов и технологий США (NIST) в августе 2024 года опубликовал первые три финализированных стандарта постквантовой криптографии: FIPS 203, FIPS 204 и FIPS 205. На российском направлении ТК 26 ведет активную работу по разработке отечественных постквантовых алгоритмов: электронной подписи "Шиповник" и "Гиперикум", а также механизма инкапсуляции ключей "Кодиеум".

Вопрос инвентаризации криптографических активов в профильной аналитике уже рассматривается как одно из ключевых узких мест постквантовой подготовки. Совместный отчёт IBM Institute for Business Value и Cloud Security Alliance *Secure the post-quantum future* фиксирует, что криптографическая инвентаризация по-прежнему остаётся эпизодической практикой: по данным опроса, лишь около 30 % организаций завершили инвентаризацию приложений, данных и сервисов с целью понять, как именно используется в них криптография, а только 24 % применяют полученные результаты для трансформации своих бизнес-процессов [1]. Авторы подчёркивают, что криптографический инвентарь должен давать «полную карту криптографических реализаций и зависимостей» и является «критически важным стартовым шагом» для разработки дорожной карты перехода к квантоустойчивым решениям, однако на практике большинство организаций не доходят до этого шага.

Непосредственный переход к алгоритмам постквантовой криптографии представляет собой масштабную инженерную задачу. Современные программные системы содержат тысячи вызовов криптографических функций, распределенных по десяткам файлов с исходным кодом. Ручная проверка подобной кодовой базы представляет собой весьма трудоемкий, подверженный ошибкам процесс, который очень плохо масштабируется на крупные проекты. В связи с этим возникает потребность в специализированном инструменте, способном автоматически обнаруживать уязвимые вызовы, оценивать их критичность в контексте конкретного программного решения, а затем формировать приоритизированный план миграции в соответствии с действующими отечественными и мировыми стандартами.

Целью данной работы является автоматизация процесса оценки и планирования миграции к постквантовым криптографическим алгоритмам посредством выявления квантово-уязвимых решений и формирования плана их модернизации на основе стандартов NIST с учетом развития отечественной нормативной базы ТК 26.

Для достижения поставленной цели должны быть решены следующие **задачи**:

- 1) Провести анализ квантовых угроз для современной криптографии, изучить профильные стандарты и выявить ограничения существующих аналогов разрабатываемого программного обеспечения.
- 2) Спроектировать модульную архитектуру инструмента и базу данных сигнатур, содержащую параметры уязвимых API-функций, форматы данных и структуру отчетов.
- 3) Реализовать программные модули инвентаризации файлов проекта и статического анализа кода для извлечения вызовов криптографических функций и анализа сертификатов.
- 4) Реализовать модули оценки рисков и планирования миграции на рекомендуемые постквантовые аналоги.
- 5) Провести функциональное тестирование на контрольной кодовой базе, проанализировать точность обнаружения.

В ходе выполнения данной работы был предложен и реализован программный инструмент, предназначенный для автоматической инвентариза-

ции криптографических активов в программных системах и оценки сопряжённых с ними квантовых рисков. Практическая значимость работы определяется тем, что инструмент позволяет без ручного вмешательства выявлять квантово-уязвимые криптографические решения в исходном коде и формировать приоритизированный план их миграции в соответствии с рекомендациями NIST и ТК 26. Личный вклад автора состоит в проектировании архитектуры инструмента, разработке базы данных криптографических сигнатур, а также в программной реализации модулей статического анализа, оценки рисков и планирования миграции.

1 АНАЛИЗ КВАНТОВЫХ УГРОЗ И НОРМАТИВНОЙ БАЗЫ

1.1 Квантовые вычисления как новый вектор угроз для современной криптографии

Современная криптографическая инфраструктура строится на предположении о вычислительной сложности определённых математических задач. Асимметричные алгоритмы – RSA, протокол Диффи–Хеллмана (DH), алгоритмы на эллиптических кривых (ECDSA, ECDH, ГОСТ Р 34.10-2012) – обязаны своей безопасностью тому, что задачи целочисленной факторизации и дискретного логарифмирования вычислительно неразрешимы на классическом компьютере за разумное время. Лучший классический алгоритм – метод решета числового поля – работает за субэкспоненциальное время порядка $e^{O((\log N)^{1/3}(\log \log N)^{2/3})}$, что делает прямые атаки практически невозможными в обозримом будущем при использовании традиционных вычислителей.

Квантовые компьютеры принципиально нарушают этот баланс. В отличие от классических машин, они оперируют кубитами – квантовыми битами, способными находиться в суперпозиции состояний. Квантовый параллелизм и явление запутанности позволяют реализовать алгоритмы, которые решают задачи факторизации и дискретного логарифмирования за полиномиальное время. На практике это означает, что весь класс задач, лежащих в основе современной асимметричной криптографии, становится критически уязвимым для достаточно мощного квантового компьютера.

Практическая реализация данной угрозы долгое время не выходила за рамки научных расчетов: для взлома RSA-2048 требовалось несколько десятков миллионов физических кубитов, что еще не так давно казалось недо-

стижимым. Однако в последние годы ситуация значительно изменилась. В декабре 2024 года компания Google представила процессор Willow со 105 свехпроводящими кубитами, впервые продемонстрировав возможность экспоненциального подавления ошибок при увеличении числа задействованных кубитов. Данный результат подтверждает принципиальную возможность создания отказоустойчивых систем, способных сохранять стабильность при масштабировании. В мае 2025 года исследователь Google Quantum AI Крейг Гидни опубликовал работу, в которой показал, что факторизация ключа RSA-2048 потенциально достижима менее чем за одну неделю на квантовом компьютере с числом физических кубитов менее одного миллиона – это в 20 раз меньше его же оценок времен 2019 года [2]. Принципиально важно, что группа из Сиднея в 2026 году опубликовала ещё более оптимистичные оценки, утверждая о возможности взлома RSA-2048 менее чем при 100 000 физических кубитов. Такого прорыва удалось достичь за счет отказа от поверхностной архитектуры коррекции ошибок в пользу квантовых LDPC-алгоритмов. Эта технология позволяет связывать кубиты в гибкую и эффективную сеть, что сокращает аппаратные затраты в десятки раз.

Согласно седьмому ежегодному докладу Global Risk Institute за 2025-2026 год [3], экспертное сообщество достигло исторического максимума в своих опасениях относительно квантовой угрозы: 26 ведущих мировых специалистов сошлись во мнении, что вероятность создания полноценного криптографически значимого квантового компьютера в ближайшее десятилетие составляет от 28% до 49%. В долгосрочной перспективе прогнозы становятся еще более определенными: на горизонте пятнадцати лет 69% участников опроса оценивают вероятность появления такой машины как 50% и выше; через 20 лет этот показатель достигает 92%. Параллельно с этим, аналитическая модель Project Eleven устанавливает конкретные сроки для наступления «Q-Day» (момента краха современной криптографии). Согласно их расчетам, наиболее вероятный сценарий реализации этой угрозы приходится на 2033 год, однако при неблагоприятном развитии событий это может произойти уже в 2030 году.

Угроза носит асимметричный характер: она нарастает не внезапно, а постепенно, и её наиболее опасный аспект разворачивается уже в данный момент – задолго до появления первого криптографически значимого кван-

тового компьютера.

Теоретические основы квантовых вычислений, включая принципы суперпозиции, запутанности и квантового параллелизма, подробно рассматриваются в работе П.Г. Ключарева «Основы квантовых вычислений и квантовой криптографии» [4]. Автор акцентирует внимание на принципиальном отличии квантовых алгоритмов от классических: если классический компьютер обрабатывает состояния последовательно, квантовый процессор в силу суперпозиции способен одновременно находиться во всех 2^n состояниях при n кубитах, что и обеспечивает экспоненциальное ускорение для определённых классов задач. Это теоретическое преимущество составляет физическую основу угроз, рассматриваемых в настоящей работе.

1.2 Алгоритм Шора: теоретическая угроза асимметричной криптографии

Алгоритм Шора был разработан американским математиком Питером Шором в 1994 году и по праву считается наиболее опасным криптоаналитическим квантовым алгоритмом из известных. Его значимость состоит в том, что он впервые предложил способ решения задачи целочисленной факторизации за полиномиальное время $O(\log^3 N)$ при использовании $O(\log N)$ кубитов, а не за субэкспоненциальное, характерное для лучших классических методов [5].

Алгоритм состоит из двух частей — классической и квантовой. Классическая часть сводит задачу факторизации числа N к задаче нахождения периода функции $f_a(x) = a^x \pmod{N}$ где a — случайное целое число, взаимно простое с N . Нахождение периода r этой функции позволяет вычислить $\gcd(a^{r/2} - 1, N)$ и $\gcd(a^{r/2} + 1, N)$, которые с высокой вероятностью окажутся нетривиальными делителями N . Именно задача нахождения периода, неэффективная на классических машинах, решается квантовой частью — с помощью квантового преобразования Фурье (QFT). QFT реализует дискретное преобразование Фурье над амплитудами квантового состояния за $O((\log N)^2)$ операций, тогда как классическое быстрое преобразование Фурье (FFT) требует $O(N \log N)$. Это и обеспечивает экспоненциальное ускорение.

Применительно к криптографии алгоритм Шора разрушает безопас-

ность всех широко используемых асимметричных систем:

- **RSA** — стойкость основана на сложности факторизации большого числа $N = p \cdot q$. Алгоритм Шора напрямую решает эту задачу.
- **ECDSA, ECDH** — стойкость основана на сложности задачи дискретного логарифмирования в группе точек эллиптической кривой. Алгоритм Шора, будучи настроен на задачу дискретного логарифмирования, решает и её за полиномиальное время.
- **ГОСТ Р 34.10-2012** — российский стандарт электронной подписи, также построенный на задаче дискретного логарифмирования в группе точек эллиптических кривых, оказывается уязвимым по той же причине. [6]
- **Схема Диффи–Хеллмана (DH/ECDH)** — протоколы обмена ключами, основанные на классической или эллиптической версии дискретного логарифмирования, также подпадают под действие алгоритма Шора.

Работоспособность алгоритма была экспериментально подтверждена в 2001 году при факторизации числа 15 на 7-кубитном квантовом компьютере. Достигнутые к 2020 году рекорды квантовой факторизации не превышали 41-битных чисел, что по-прежнему бесконечно далеко от RSA-2048. Однако принципиальным является не нынешнее состояние, а вектор развития. Снижение оценочного числа физических кубитов для взлома RSA-2048 от 20 миллионов в 2019 году до менее чем 1 миллиона в 2025 году демонстрирует, что ресурсные барьеры стремительно уменьшаются.

Особую уязвимость проявляют системы на эллиптических кривых. Исследования показывают, что для взлома 256-битного ключа ECDSA требуется примерно 2 330 логических кубитов, тогда как для взлома RSA-2048 — порядка 4 098, а по последним оценкам алгоритмические улучшения снизили нижнюю границу для RSA-2048 до 1 730 логических кубитов. Таким образом, ECC-системы могут оказаться уязвимыми раньше RSA-систем при одинаковой оценке угрозы.

Критично понимать, что алгоритм Шора не затрагивает симметричные алгоритмы (AES, «Кузнечик») и хэш-функции (SHA, «Стрибог»), поскольку их безопасность не основана на задачах факторизации или дискретного логарифмирования. Однако, поскольку в современных протоколах симметричное

шифрование неизменно применяется в связке с асимметричным (например, в TLS асимметрика используется для безопасного распределения симметричного сеансового ключа), компрометация асимметричного слоя фактически разрушает всю цепочку защиты.

1.3 Алгоритм Гровера: влияние на симметричную криптографию

Алгоритм Гровера, предложенный Ловом Гровером в 1996 году, представляет собой квантовый алгоритм поиска в неструктурированных базах данных. В контексте криптоанализа он применим к задаче перебора ключей симметричных шифров: вместо того чтобы перебирать все 2^n возможных ключей, квантовый вычислитель с помощью алгоритма Гровера способен найти ключ за $O(2^{n/2})$ операций. Иными словами, алгоритм обеспечивает квадратичное ускорение перебора, что эквивалентно уменьшению эффективной длины ключа вдвое.

Практические следствия для современных симметричных алгоритмов выглядят следующим образом:

- **AES-128** — эффективная безопасность снижается с 128 бит до приблизительно 64 бит, что соответствует уровню, считающемуся недостаточным по современным стандартам.
- **AES-256** — эффективная безопасность снижается со 256 бит до 128 бит, что по-прежнему признаётся достаточным уровнем защиты.
- **AES-192** — промежуточный вариант с эффективной безопасностью порядка 96 бит, который также принято считать приемлемым.

NIST и АНБ США в своих рекомендациях прямо указывают, что AES-256 остаётся безопасным даже в присутствии квантового противника, реализующего алгоритм Гровера. Именно поэтому основной рекомендацией является не полная замена симметричных алгоритмов, а переход к вариантам с большей длиной ключа — в первую очередь к AES-256.

Вместе с тем необходимо учитывать ряд существенных ограничений алгоритма Гровера как практической угрозы. В отличие от алгоритма Шора, алгоритм Гровера фундаментально последователен: каждая следующая итерация зависит от результата предыдущей, что делает его практически нераспараллеливаемым. Анализ, проведённый в 2026 году Флориано Вальсорта,

показывает, что эксплуатация алгоритма Гровера в масштабе, достаточном для взлома AES-128, потребовала бы нереалистично большого числа квантовых систем и вычислительных ресурсов. Атака на AES-128, теоретически требующая 2^{64} квантовых шагов, при последовательном исполнении заняла бы сотни тысяч лет. Параллельное выполнение не улучшает совокупную стоимость атаки, поскольку суммарное число операций возрастает пропорционально числу параллельных экземпляров.

Авторитетная работа, опубликованная в журнале Nature Scientific Reports в 2024 году, исследует практическую стоимость применения алгоритма Гровера к схемам AEAD на основе AES, подчёркивая, что реальные ресурсные требования значительно превышают теоретические оценки [7]. NIST в рамках документа NIST IR 8547 явным образом не устанавливает требования по смене длины ключей симметричных алгоритмов в связи с угрозой алгоритма Гровера [8], а симметричные алгоритмы с безопасностью 128 бит и выше не включены в перечень алгоритмов, подлежащих устареванию.

Таким образом, угроза алгоритма Гровера для симметричной криптографии является умеренной и управляемой: она имеет больше значения для долгосрочного планирования, чем для немедленных действий. Реальный приоритет постквантового перехода сосредоточен на асимметричных алгоритмах, где угроза носит качественный, а не количественный характер.

1.4 Угроза «перехвати сейчас – расшифруй потом» (HN DL)

Одним из наиболее опасных и практически значимых аспектов квантовой угрозы является стратегия, получившая название «Harvest Now, Decrypt Later» (HN DL) — «перехвати сейчас, расшифруй потом», также известная как «Store Now, Decrypt Later» (SN DL). Суть стратегии состоит в том, что атакующие уже сегодня перехватывают и архивируют зашифрованные данные, защищённые классическими криптографическими алгоритмами (RSA, ECC), с намерением расшифровать их в будущем — когда у них появится доступ к достаточно мощному квантовому компьютеру.

Именно HN DL превращает квантовую угрозу из гипотетической будущей проблемы в актуальный риск настоящего момента. Любые данные, перехваченные и сохранённые сегодня, будут оставаться под угрозой до того дня, пока не появится CRQC — вне зависимости от того, когда организация-

жертва выполнит миграцию на постквантовые алгоритмы. Миграция закрывает угрозу для будущих коммуникаций, но никак не защищает данные, уже находящиеся в архивах противника.

Атака HNDL разворачивается в три этапа:

- 1) **Перехват** — злоумышленник собирает зашифрованный трафик с помощью традиционных методов: прослушивания сетей, компрометации конечных точек или кражи резервных копий. Поскольку данные зашифрованы, немедленная расшифровка не требуется.
- 2) **Хранение** — перехваченные данные архивируются на долгий срок — годы или десятилетия — в ожидании технологического прорыва. Этот этап практически не поддаётся обнаружению.
- 3) **Расшифровка** — при появлении CRQC накопленный архив данных расшифровывается с помощью алгоритма Шора. Итогом становится не внезапная атака, а отложенное раскрытие данных, похищенных задолго до этого момента.

Особую уязвимость перед HNDL представляют долгоживущие данные: государственные секреты, медицинские записи, финансовые транзакции, интеллектуальная собственность и данные национальной безопасности, которые требуют конфиденциальности на горизонте десятилетий. Британский Национальный центр кибербезопасности (NCSC) в своём годовом обзоре 2023 года констатировал, что государственные акторы уже ведут кампании по краже данных «для использования в будущем». АНБ США ещё в 2021 году предупредило о том, что противники активно собирают зашифрованные данные, предвосхищая появление квантовых вычислителей.

Именно угроза HNDL является основным обоснованием срочности перехода к постквантовой криптографии во всех ключевых нормативных документах — директивах NIST, рекомендациях АНБ (CNSA 2.0) и руководстве Евросоюза. Организации, обрабатывающие данные с длительным сроком конфиденциальности, должны оценить так называемое «окно HNDL-уязвимости»: временной интервал, в течение которого данная категория данных оставалась зашифрованной классическими алгоритмами, и сравнить его с прогнозируемым сроком появления CRQC. Если этот интервал перекрывает горизонт Q-Day, данные следует считать потенциально скомпрометированными уже сейчас.

1.5 Стандарты NIST в области постквантовой криптографии

Признание квантовой угрозы заставило Национальный институт стандартов и технологий США (NIST) ещё в декабре 2016 года инициировать масштабный открытый конкурс по стандартизации алгоритмов постквантовой криптографии. К ноябрю 2017 года было подано 82 заявки, из которых 69 прошли первичный отбор и были приняты в первый раунд [9]. По итогам трёх последовательных раундов оценки, в каждом из которых число кандидатов сокращалось на основании криптоанализа, исследований безопасности и практических показателей, NIST в 2022 году объявил о четырёх алгоритмах-кандидатах: CRYSTALS-Kyber, CRYSTALS-Dilithium, Falcon и SPHINCS+.

13 августа 2024 года NIST опубликовал три первых финализированных стандарта постквантовой криптографии:

FIPS 203 (ML-KEM, Module-Lattice-Based Key-Encapsulation Mechanism) — основной стандарт инкапсуляции ключей, разработанный на основе алгоритма CRYSTALS-Kyber. Безопасность алгоритма основана на вычислительной сложности задачи обучения с ошибками на решётках (Module-LWE). Стандарт предусматривает три набора параметров:

- ML-KEM-512 — категория безопасности 1 (эквивалент AES-128);
- ML-KEM-768 — категория безопасности 3 (эквивалент AES-192);
- ML-KEM-1024 — категория безопасности 5 (эквивалент AES-256).

FIPS 204 (ML-DSA, Module-Lattice-Based Digital Signature Algorithm) — основной стандарт цифровой подписи, разработанный на основе алгоритма CRYSTALS-Dilithium. Как и ML-KEM, безопасность опирается на задачу Module-LWE. Предусмотрены варианты параметров ML-DSA-44, ML-DSA-65 и ML-DSA-87, соответствующие категориям безопасности 2, 3 и 5 соответственно.

FIPS 205 (SLH-DSA, Stateless Hash-Based Digital Signature Algorithm) — альтернативный стандарт цифровой подписи на основе алгоритма SPHINCS+. В отличие от двух предыдущих, безопасность которых основана на задачах теории решёток, SLH-DSA опирается исключительно на стойкость криптографических хэш-функций, что обеспечивает диверсификацию математических оснований. Стандарт включает 12 вариан-

тов параметров в сочетании функций SHA2/SHAKE с приоритетом скорости или компактности подписи.

Помимо этого, в четвёртом раунде конкурса был отобран алгоритм HQC, основанный на теории кодов, исправляющих ошибки, который был выбран для стандартизации в марте 2025 года как дополнение к алгоритмам на решётках. Ранее планировался к финализации FIPS 206 на основе алгоритма Falcon для приложений, требующих компактного размера подписи.

Параллельно с публикацией стандартов NIST в ноябре 2024 года выпустил NIST IR 8547 — концепцию временных рамок перехода. Документ устанавливает следующие ключевые сроки:

- **2030 год** — устаревание алгоритмов с безопасностью порядка 112 бит: RSA-2048 и ECDSA P-256 объявляются нерекомендуемыми для новых систем.
- **2035 год** — полный запрет использования любых квантово-уязвимых алгоритмов с открытым ключом в стандартах NIST и документах FIPS, включая RSA с любой длиной ключа и ECC на любых кривых.

Отдельно АНБ США в рамках CNSA 2.0 (Commercial National Security Algorithm Suite 2.0), опубликованного в сентябре 2022 года, установило жёсткие сроки для систем национальной безопасности: обязательная поддержка алгоритмов ML-KEM-1024 и ML-DSA-87 для новых систем — немедленно, для всех систем национальной безопасности — к 2030 году, исключительное использование только CNSA 2.0 — к 2033 году. NIST прямо рекомендует организациям начать развёртывание PQC-алгоритмов немедленно, не дожидаясь финализации всех стандартов и нормативных документов.

1.6 Рекомендации ТК 26 в контексте постквантового перехода

На российском нормативном направлении координирующую роль в области разработки постквантовых стандартов выполняет Технический комитет по стандартизации «Криптографическая защита информации» (ТК 26) Росстандарта. Ведущая роль в комитете принадлежит Федеральной службе безопасности (ФСБ России) как основному отраслевому регулятору. Внутри ТК 26 на постоянной основе действуют рабочие группы из представителей ведущих компаний российского рынка информационной безопасности.

В 2019 году была создана рабочая группа «Постквантовые криптографические механизмы», в состав которой вошли три подгруппы по направлениям: постквантовые механизмы на основе хэш-функций, на основе кодов, исправляющих ошибки, и на основе теории целочисленных решёток. Ключевую экспертную роль в деятельности всех подгрупп играют специалисты компаний QApp и «Криптонит».

К 2024–2025 годам в рамках рабочей группы были разработаны и представлены следующие алгоритмы-кандидаты:

- **«Шиповник»** — схема электронной подписи, основанная на теории кодов, исправляющих ошибки (разработчик — НПК «Криптонит»). Алгоритм прошёл первый этап научно-технической экспертизы в ТК 26. Схема рассматривается как потенциальная замена ГОСТ Р 34.10-2018.
- **«Гиперикум»** — схема электронной подписи, основанная на хэш-функциях, разработанная компанией QApp на основе экспертизы, полученной при анализе международного алгоритма SPHINCS+. Алгоритм был представлен в апреле 2023 года и находится в процессе стандартизации.
- **«Кодиеум»** — схема инкапсуляции ключей, основанная на кодах Гоппы — широком классе кодов, разработанных советским учёным В.Д. Гоппой. Алгоритм разработан совместно специалистами «Криптонита» и ТК 26, представлен на конференции «РусКрипто'24» в марте 2024 года, рассматривается как постквантовая альтернатива протоколу Диффи–Хеллмана.

Отечественные разработки в области постквантовых схем охватывают не только алгоритмы электронной подписи и механизмы инкапсуляции ключей, но и смежные криптографические примитивы. В частности, применимость существующих схем разделения секрета в постквантовых условиях исследована в работе Кустова Е.Ф. и Беззатеева С.В. «Анализ применимости существующих схем разделения секрета в условиях постквантовой эры» [10]. Авторы показывают, что классические схемы Шамира, основанные на интерполяции многочленов в конечных полях, остаются вычислительно эффективными, однако уязвимы к квантовым атакам, тогда как постквантовые схе-

мы на основе теории решёток и кодов, исправляющих ошибки, обеспечивают необходимый уровень стойкости, но требуют значительно более сложных вычислений. Это наблюдение актуально для задач настоящей работы: разрабатываемый инструмент должен учитывать не только прямые вызовы асимметричных примитивов, но и криптографические схемы управления ключами, реализованные поверх них.

Принципиальное отличие российского подхода от конкурса NIST состоит в том, что ТК 26 не проводит конкурсного отбора победителей из множества кандидатов, а ведёт параллельную разработку сразу нескольких механизмов различной математической природы. Все разрабатываемые алгоритмы в перспективе должны дополнить комплекс стандартов серии ГОСТ Р 34, не заменяя действующих стандартов до завершения полноценной стандартизации.

Важно понимать, что на момент написания настоящей работы российские постквантовые алгоритмы находятся на стадии разработки и экспертизы: ни один из них ещё не получил статус государственного стандарта. Это создаёт специфическую нормативную ситуацию: организации, планирующие постквантовый переход в российской юрисдикции, вынуждены опираться преимущественно на международные стандарты NIST при отсутствии отечественной нормативной базы. Тем не менее ТК 26 в своих методических материалах признаёт актуальность угрозы и указывает на необходимость уже сейчас планировать переход на постквантовые алгоритмы, не ожидая финализации отечественных стандартов. По словам представителей «КриптоНИИ», разработка и внедрение новых криптографических механизмов занимает не годы, а десятилетия, в связи с чем необходимо начинать работу немедленно.

2 АНАЛИЗ СУЩЕСТВУЮЩИХ АНАЛОГОВ

Прежде чем приступить к проектированию разрабатываемого инструмента, необходимо дать ответ на вопрос: почему задача автоматической инвентаризации квантово-уязвимой криптографии в C/C++ кодовых базах до сих пор не решена существующими средствами? Для этого нужно рассмотреть наиболее известные инструменты, в явном виде претендующие на роль аналогов, и оценить каждый из них через призму конкретных требова-

ний: статический анализ исходного кода на языках C и C++, обнаружение квантово-уязвимых криптографических вызовов (включая вызовы OpenSSL, Crypto++ и ГОСТ-совместимых библиотек), оценка квантового риска и формирование приоритизированного плана миграции с учётом требований NIST IR 8547 и отечественной нормативной базы ТК 26.

2.1 IBM Quantum Safe (экосистема)

IBM строит своё предложение в сфере постквантовой безопасности как многоуровневую экосистему, охватывающую полный жизненный цикл перехода: от обнаружения уязвимостей в исходном коде до стратегического управления миграцией на уровне всего предприятия. Компания напрямую участвовала в разработке трёх из четырёх финализированных алгоритмов NIST и ведёт практическую работу по постквантовому переходу с корпоративными клиентами с 2019 года. Экосистема состоит из пяти взаимосвязанных компонентов, каждый из которых соответствует определённому этапу цикла.

IBM Quantum Safe Explorer отвечает за первый, наиболее трудоёмкий этап — статический анализ исходного кода и формирование криптографического инвентаря. Инструмент поддерживает анализ кода на Java, C, C++, Python, Dart, Go и C#, однако при этом существует принципиальное технологическое разграничение между поддерживаемыми языками. Для Java реализован режим «Cryptography Analysis» — полноценный анализ с межметодной трассировкой переменных, то есть инструмент способен определить конкретный алгоритм, переданный через несколько уровней вызовов в качестве строкового параметра, а также выявить антишаблоны крипто-гибкости: жёстко закодированные имена алгоритмов, несогласованное использование библиотек и отсутствие механизмов резервного согласования. Для всех остальных языков — включая C и C++ — доступен только режим «API Discovery»: инструмент фиксирует сам факт вызова криптографической функции и её местоположение в коде, но не прослеживает аргументы и не восстанавливает семантику вызова. Это принципиальное различие хорошо иллюстрируется характерным паттерном кода на C:

В режиме API Discovery инструмент зафиксировывает вызов *EVP_CipherInit_ex*, однако не сможет определить, какой именно ал-

```
const char *algo = get_config("cipher");  
EVP_CIPHER_CTX *ctx = EVP_CipherInit_ex(NULL, EVP_get_cipherbyname(algo), ...);
```

Рисунок 2.1 – Паттерн кода на C

горитм в него передаётся. Именно здесь — в неспособности восстановить криптографическую семантику применительно к C/C++ — заключается ключевое техническое ограничение Explorer. Для C и C++ поддерживаются библиотеки OpenSSL (libCrypto и libSSL), Crypto++, GSKit-crypto и liboqs; российские криптографические библиотеки в этот список не входят.

Тем не менее Explorer обеспечивает значимую функциональность даже в режиме API Discovery: итогом сканирования является CBOM (Cryptography Bill of Materials) в форматах CycloneDX, CSV и JSON.

IBM Quantum Safe Advisor решает задачу, которую статический анализ в принципе не может решить: он анализирует криптографическую среду в режиме реального времени. Advisor исследует активные TLS-сессии, развёрнутые сертификаты X.509, задействованные шифронаборы и потоки данных между активами, выстраивая операционную картину того, как криптография используется непосредственно в процессе эксплуатации — картину, принципиально недостижимую при статическом анализе кода. Ключевое добавление Advisor к данным Explorer — AI-приоритизация: компонент оценивает чувствительность данных, защищаемых каждым активом, и на основании этого ранжирует активы по срочности миграции. Непрерывный мониторинг, в отличие от разовых «снимков» Explorer, фиксирует изменения в криптографической среде и позволяет отслеживать регрессии после выполненных изменений.

IBM Quantum Safe Remediator является единственным компонентом экосистемы, непосредственно участвующим в реализации перехода. Технологически он реализует концепцию «Adaptive Proxy» — сетевого прокси, который встаёт перед legacy-приложением и прозрачно переводит TLS-трафик на постквантовые алгоритмы (ML-KEM, ML-DSA, Falcon) без какого-либо вмешательства в исходный код приложения. Прокси поддерживает три режима обработки TLS-трафика: Edge (завершение соединения на прокси и новое соединение к бэкенду с PQC), Re-encrypt (дешифровка и повторное шифрование с использованием сертификата бэкенда) и Passthrough (пере-

направление на транспортном уровне без расшифровки). Существенно, что прокси обеспечивает обратную совместимость, одновременно обслуживая как legacy-клиентов с классическими шифронаборами, так и PQC-клиентов, что позволяет выполнять переход постепенно. Помимо прокси, Remediator включает Performance Harness — инструмент для сравнительного бенчмаркинга производительности TLS при legacy, гибридных и полностью PQC-конфигурациях, позволяющий оценить деградацию производительности до развёртывания в production.

IBM Quantum Safe Migration Orchestrator (QSMO) закрывает уровень стратегического управления переходом. Анонсированный в декабре 2025 года и доступный исключительно через IBM Consulting, он выполняет многомерную приоритизацию рисков с учётом бизнес-контекста каждого актива, формирует многогоризонтную дорожную карту с балансировкой срочности, реализуемости и соответствия стандартам, интегрируется с CMDB, CBOM, PKI, CLM и KMS для превращения плана в выполняемые действия. Дашборды и отчёты готовности к аудиту ориентированы на уровень CISO и совета директоров.

IBM Guardium Cryptography Manager, представленный в октябре 2025 года, дополняет экосистему управлением жизненным циклом ключей и сертификатов: он автоматизирует ротацию ключей, обнаружение незашифрованных данных в базах и реализует последовательность действий для непрерывного перехода на постквантовые стандарты.

В рамках настоящей работы важнее всего зафиксировать, что экосистема IBM в целом покрывает полный функциональный цикл постквантового перехода, однако это покрытие носит строго ограниченный характер. C/C++ кодовые базы получают только поверхностный API Discovery без криптографической семантики; российские библиотеки и алгоритмы ТК 26 не поддерживаются ни одним компонентом; полный цикл поддержки требует корпоративного контракта с IBM.

2.2 CodeQL

CodeQL — статический анализатор кода, разработанный компанией Semmle и приобретённый GitHub в 2019 году, сегодня являющийся основой встроенного сканирования Code Scanning в GitHub Actions. Архитектур-

но CodeQL принципиально отличается от всех остальных рассматриваемых инструментов: вместо прямого анализа исходного кода он сначала компилирует его в реляционную базу данных (CodeQL database), которая представляет программу в виде набора отношений — типов, переменных, вызовов, потоков управления и данных, — а затем позволяет выполнять по этой базе запросы на языке QL, синтаксически похожем на SQL. Это даёт CodeQL уникальную возможность: запросы способны проследивать потоки данных через произвольное число уровней вызовов и модулей, что недоступно ни одному из инструментов, работающих на уровне AST-паттернов.

Для задачи постквантового аудита CodeQL представляет особый интерес по нескольким причинам. В декабре 2023 года команда исследователей GitHub опубликовала набор кастомных QL-запросов, реализующих CBOM-генерацию. Авторы определили открытую модель криптографических абстракций в CodeQL — абстрактные классы, представляющие алгоритм, режим, длину ключа и контекст использования, — и расширили эти абстракции для конкретных API OpenSSL, JCA и других библиотек. Полученные запросы применялись в режиме Multi-Repository Variant Analysis (MRVA), позволяющем запускать один запрос одновременно против до 1 000 репозиторий на GitHub. CodeQL поддерживает полноценный межпроцедурный — analysis для C и C++: запрос способен отследить, как строковое значение, считанное из конфигурационного файла, передаётся через несколько функций и в конечном счёте становится аргументом алгоритма шифрования в OpenSSL.

Всё это делает CodeQL концептуально наиболее мощным инструментом для глубокого криптографического анализа C/C++ кодовых баз. Однако именно эта мощь несёт с собой критические практические ограничения. CodeQL требует успешной компиляции анализируемого кода: для C и C++ нет возможности проанализировать произвольный репозиторий без корректно настроенной системы сборки. Для реальных проектов это означает необходимость наличия работоспособного CMakeLists.txt, Makefile или compile_commands.json, что существенно усложняет развёртывание инструмента в сценариях быстрого аудита или при работе с legacy-кодовыми базами с неактуальной системой сборки. Кроме того, готовые постквантовые запросы для C/C++ в публичных репозиториях на сегодняшний день значительно менее полны, чем для Java: опубликованные командой GitHub запросы в

первую очередь ориентированы на Java и требуют существенной доработки для покрытия полного спектра OpenSSL API, характерного для С-кода. Написание и поддержка кастомных QL-запросов требуют специфических знаний языка запросов, что создаёт значительный порог входа. Наконец, CodeQL не генерирует оценку квантового риска, не производит сопоставления с требованиями NIST IR 8547 и не формирует рекомендаций по замене алгоритмов или плана миграции — он остаётся инструментом инвентаризации, пусть и исключительно мощным.

2.3 CBOMkit

CBOMkit — открытый инструментальный комплекс для работы с Cryptography Bill of Materials, разработанный исследовательским подразделением IBM Research и переданный в 2025 году под управление Альянса постквантовой криптографии (Post Quantum Cryptography Alliance, PQCA) при Фонде Linux. Это, по существу, открытая версия технологии, лежащей в основе IBM Quantum Safe Explorer, однако без корпоративной оболочки и с более прозрачной архитектурой.

В основе экосистемы CBOMkit лежит плагин CBOMkit-hyperion — плагин для SonarQube, который анализирует исходный код на уровне абстрактного синтаксического дерева (AST) и формирует CBOM-документы с точным указанием файла, строки и контекста каждой находки. Плагин поддерживает Java и Python; в апреле 2026 года была добавлена официальная поддержка Go. Помимо анализатора кода, экосистема включает CBOMkit-theia — инструмент для сканирования образов контейнеров Docker/OCI на предмет криптографических артефактов в конфигурациях безопасности, сертификатах и секретах; CBOMkit-coeus — веб-интерфейс для визуализации результатов; CBOMkit-mnemosyne — централизованное хранилище с RESTful API для агрегации результатов по командам и репозиториям; и CBOMkit-themis — движок проверки соответствия политикам, который оценивает сгенерированный CBOM на соответствие настроенным правилам.

Компонент Themis — единственный в составе CBOMkit, приближающийся к функциональности оценки постквантовой готовности. Встроенная служба BasicQuantumSafeComplianceService сверяет обнаруженные алгоритмы с белым списком квантово-безопасных OID и имён алгоритмов: активы,

не входящие в список, помечаются как несоответствующие. Таким образом, Themis отвечает на вопрос «соответствует ли данный актив политике?», но не генерирует рекомендаций о том, на что именно следует заменить несоответствующий алгоритм, и не формирует план перехода. Эксперт IBM Research Майкл Осборн разграничивает два принципиально разных типа СВОМ: «инвентаризационный» — отвечающий на вопрос «что есть?» — и «миграционный», отвечающий на вопросы «что должно измениться, кто это координирует и что блокирует переход?». СВОМkit реализует первый тип, тогда как второй, предполагающий включение в документ конфигурации каждого актива в runtime, цепочек зависимостей и roadmap вендоров, по-прежнему находится за рамками возможностей инструмента.

Применительно к задаче настоящей работы ключевым ограничением СВОМkit является отсутствие стабильной поддержки C и C++. В феврале 2026 года разработчик из сообщества объявил о готовности реализации C/C++ поддержки на основе плагина sonar-cxx с покрытием правил для OpenSSL libCrypto и libSSL, однако эта реализация ещё не влита в основную ветку проекта. Межметодное отслеживание переменных отсутствует для всех поддерживаемых языков: обнаружение работает только в рамках одного метода. Российские библиотеки криптографической защиты информации не поддерживаются.

2.4 CryptoScan

CryptoScan — открытый инструмент криптографического сканирования, разработанный в рамках проекта QRAMM (Quantum Risk Assessment, Migration, and Management). Из всех рассматриваемых open-source аналогов он, пожалуй, наиболее функционально близок к задаче разрабатываемого инструмента: в отличие от общих SAST-платформ, CryptoScan создавался специально для постквантового аудита с самого начала.

Инструмент реализует обнаружение более 50 криптографических паттернов: RSA, ECDSA, классический DH, AES, DES, MD5, SHA-1, конфигурации TLS, приватные ключи и другие артефакты. Каждая находка классифицируется по уровню квантового риска — это принципиальное отличие от большинства аналогов, которые лишь фиксируют факт использования алгоритма, не давая оценки степени угрозы. По итогам сканирования генериру-

ются отчёты в форматах SARIF (для интеграции с GitHub Security), SBOM, JSON и HTML. SARIF-вывод позволяет использовать результаты сканирования непосредственно в интерфейсах GitHub Code Scanning и других системах управления уязвимостями. Инструмент поддерживает интеграцию в CI/CD и умеет блокировать сборку при обнаружении критически важных уязвимостей.

Экосистема QRAMM дополняет CryptoScan двумя смежными инструментами. TLS Analyzer анализирует конфигурации TLS/SSL на соответствие CNSA 2.0, оценивая шифронаборы, протоколы и сертификаты в контексте постквантовых сроков. CryptoDeps сканирует зависимости проекта (пакеты Go, npm, Python/PyPI, Maven) на предмет квантово-уязвимых алгоритмов, генерируя SBOM с оценкой рисков по транзитивным зависимостям. Совместное применение трёх инструментов позволяет покрыть три разных вектора обнаружения: исходный код, сетевая инфраструктура и цепочка поставок.

Однако применительно к C/C++ кодовым базам CryptoScan имеет существенное ограничение: список поддерживаемых языков на официальной документации не конкретизирован, а примеры на сайте ориентированы преимущественно на скриптовые языки и Go. GitHub-репозиторий проекта указывает на детектирование 15+ языков, однако конкретные механизмы анализа C/C++ (уровень AST, regex-паттерны или иное) не документированы. Глубина анализа C/C++ кода остаётся неизвестной и, по всей видимости, ограничивается текстовым поиском по паттернам, что не позволяет корректно обрабатывать вызовы, где имя алгоритма передаётся через переменную или макрос. Рекомендации по конкретным постквантовым аналогам не генерируются, план миграции отсутствует. Поддержка российских криптографических библиотек не предусмотрена.

2.5 SonarQube

SonarQube — платформа непрерывного статического анализа кода с открытым ядром, разработанная компанией SonarSource и широко применяемая в DevSecOps-практиках. В контексте постквантовой тематики SonarQube релевантен в следующем смысле: как самостоятельный инструмент со встроенными правилами безопасности для C/C++ и как инфраструктура, на которой строится плагин SBOMkit-hyperion (sonar-cryptography).

Нативные возможности SonarQube применительно к С и С++ в части безопасности криптографии включают семь встроенных правил: обнаружение слабых алгоритмов шифрования (DES, RC4, MD5), небезопасных режимов и схем паддинга, ненадёжных криптографических ключей, отключённой проверки серверных сертификатов и слабых версий SSL/TLS. Все эти правила доступны как для С, так и для С++ начиная с версии SonarQube 8.6 без установки дополнительных плагинов. Анализ С++ выполняется в режиме AutoConfig, не требующем ручной настройки компиляционных баз данных, что снижает барьер входа по сравнению с CodeQL. Механизм анализа помеченных данных позволяет отслеживать потоки данных от источников до уязвимых функций в пределах одного метода.

Вместе с тем нативные правила SonarQube для С/С++ выявляют общие небезопасные практики, но принципиально не ориентированы на постквантовый контекст. Ни одно из правил не идентифицирует вызовы RSA, ECDSA или классического DH как квантово-уязвимые: платформа сигнализирует о применении алгоритмов, небезопасных в классическом смысле (слабые по размеру ключа и т. п.), тогда как RSA-2048 с точки зрения классических угроз остаётся безопасным в использовании. Разрыв между «небезопасно сейчас» и «небезопасно в квантовом будущем» платформой не различается. Для задачи постквантового аудита постоянно используемые, корректно реализованные RSA и ECDSA-вызовы не будут помечены как потенциально проблемные.

Постквантовая функциональность достигается только через плагин sonar-cryptography (CBOMkit-hyperion), однако, как было отмечено в разделе 2.3, поддержка С/С++ реализуется усилиями сообщества и официально ещё не поддерживается. Помимо этого, плагин не является официальным продуктом SonarSource, и его совместимость с текущими версиями SonarQube требует отдельной проверки.

Таким образом, SonarQube в качестве инструмента постквантового аудита С/С++ требует дополнительной установки внешних, не прошедших проверку стабильности компонентов и не покрывает ни оценку квантового риска, ни планирование полноценной миграции.

2.6 CycloneDX cdxgen

CycloneDX cdxgen — официальный генератор биллей материалов в экосистеме OWASP CycloneDX, поддерживаемый Фондом OWASP. Инструмент занимает особое место в ландшафте аналогов: его задача не анализ криптографических вызовов как таковых, а генерация человекочитаемых документов, описывающих состав программной системы, в том числе её криптографических компонентов, для последующей передачи в системы управления рисками.

cdxgen поддерживает генерацию пяти типов BOM в рамках одного рабочего потока: SBOM (программные компоненты), CBOM (криптографические активы), OBOM (операционная среда), SaaSOM и документы аттестации. Уникальной возможностью инструмента является анализ достижимости кода (reachability analysis) для Java, JavaScript и TypeScript: используя AST-инструмент atom, cdxgen определяет, реально ли тот или иной криптографический вызов выполняется в рамках кодовых путей, что снижает уровень ложноположительных срабатываний. CBOM-документ в формате CycloneDX 1.6 может быть передан в систему Dependency-Track для непрерывного мониторинга и приоритизации.

Применительно к задаче настоящей работы cdxgen имеет два существенных ограничения. Во-первых, CBOM-генерация поддерживается только для Java и Python: для C и C++ инструмент способен генерировать SBOM при наличии поддерживаемого пакетного менеджера (например, Conan), однако криптографические вызовы при этом не анализируются и в CBOM не включаются. Во-вторых, cdxgen не производит никакой оценки квантового риска: он фиксирует наличие криптографических алгоритмов в коде, но не различает, является ли тот или иной алгоритм квантово-уязвимым и насколько срочна его замена. Оценка и приоритизация рисков делегируются Dependency-Track или иным внешним системам — если те поддерживают постквантовую семантику. Таким образом, cdxgen является ценным инструментом в составе более широкого пайплайна, но не самостоятельным решением задачи постквантового аудита C/C++ кодовых баз.

2.7 QApp

QApp — российская компания, специализирующаяся на постквантовой криптографии, — представляет особый интерес в контексте данной работы не как инструмент статического анализа кода, а как единственный отечественный игрок, одновременно активно участвующий в формировании российских постквантовых стандартов и предлагающий практические решения для необходимого перехода.

Компания участвует во всех трёх подгруппах рабочей группы ТК 26 «Постквантовые криптографические механизмы» и является разработчиком алгоритма электронной подписи «Гиперикум». В марте 2024 года алгоритм «Гиперикум», построенный на основе хэш-функций и разработанный с опорой на экспертизу в области международного алгоритма SPHINCS+, был представлен на конференции «РусКрипто» в рамках ТК 26. Эта роль предоставляет QApp уникальную осведомлённость об отечественном нормативном контексте, недоступную ни одному из международных аналогов.

Основным продуктом компании является PQLR SDK — переносимая библиотека квантово-устойчивых алгоритмов с интеграцией в OpenSSL, Java JCA и Android Conscrypt. SDK предоставляет реализации актуальных PQС-алгоритмов (ML-KEM/Kyber, ML-DSA/Dilithium и другие) и ориентирован на встраивание в существующие приложения. В 2023 году совместно с НСПК (Национальной системой платёжных карт) был реализован пилотный проект по защите системы электронного документооборота с использованием постквантовых алгоритмов через QTunnel — продукт компании для квантово-устойчивых защищённых каналов связи.

Вместе с тем QApp не предлагает публично доступного инструмента для автоматического статического анализа произвольных кодовых баз на предмет квантовой уязвимости. Поддержка миграции реализована в форме профессиональных услуг и SDK: предполагается, что организация либо самостоятельно определила перечень компонентов, требующих замены, либо привлекает инженеров QApp для проведения аудита. Пилот с НСПК наглядно демонстрирует этот подход: переход выполнялся в рамках совместного проекта с активным участием команды компании, а не посредством самостоятельного применения инструмента заказчиком. Это принципиально иная бизнес-модель по сравнению с инструментами, рассмотренными выше.

2.8 Выводы по результатам анализа аналогов

Анализ семи инструментов позволяет установить печальную закономерность: каждый из них оптимизирован под один из этапов цикла постквантового перехода, но ни один из них не реализует полную цепочку «обнаружение квантово-уязвимых C/C++ вызовов → оценка риска → рекомендации по конкретным постквантовым аналогам → приоритизированный план с учётом нормативных сроков».

Наиболее показателен провал в области C и C++ — языков, на которых реализована значительная часть системного программного обеспечения, встраиваемых систем, криптографических библиотек и промышленных приложений. IBM Quantum Safe Explorer предоставляет лишь поверхностное API Discovery без трассировки аргументов, а значит не позволяет определить, какой именно алгоритм используется в типичном C-коде. Технически CodeQL способен выполнить полноценный анализ для C/C++, однако требует для этого обязательной компиляции исходного кода. Это делает его неприемлемым для быстрого аудита произвольных репозиториями. Кроме того, он не содержит готовых постквантовых правил. SBOMkit не имеет стабильного релиза с поддержкой C/C++. CryptoScan не документирует глубину анализа C/C++. SonarQube обнаруживает классически небезопасные алгоритмы, но не квантово-уязвимые. Cdxgen полностью исключает C/C++ из SBOM-анализа.

Дополнительно, все международные инструменты игнорируют российскую нормативную специфику: библиотеки, реализующие ГОСТ Р 34.10-2012, «Кузнечик» и «Стрибог», а также разрабатываемые алгоритмы ТК 26 не входят в базы сигнатур ни одного из них. Для программных систем, работающих в российской юрисдикции и использующих ГОСТ-совместимые технологии, применение данных инструментов не дает полноценного результата.

Наконец, ни один из открытых инструментов не производит сопоставления обнаруженных алгоритмов с нормативными сроками NIST IR 8547: инструмент, обнаруживший RSA-2048, не сообщает пользователю, что данный алгоритм должен быть выведен из использования в системах, создаваемых после 2030 года, и запрещён к применению к 2035 году. Это разрывает связь между технической находкой и организационным решением о приоритете за-

мены.

3 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ИНСТРУМЕНТА

3.1 Формирование требований

После анализа предметной области и существующих решений была сформулирована задача проектирования инструмента, который не ограничивается простым поиском названий криптографических алгоритмов в исходном коде, а выполняет полный цикл обработки: инвентаризирует структуру проекта, обнаруживает квантово-уязвимые криптографические активы, анализирует сертификаты, оценивает риски, формирует план миграции и сохраняет результат в человекочитаемом виде. При проектировании было важно не смешивать эти операции в одном монолитном модуле, поскольку каждая из них имеет собственные входные данные, собственные ограничения и собственный формат результата.

Разработка требований началась с выделения основных объектов, с которыми должен работать инструмент. В качестве первого объекта рассматривался сам анализируемый проект: его файловая структура, исходные файлы, заголовочные файлы, тесты, файлы сборки и сертификаты. Вторым объектом являлись криптографические вызовы в исходном коде. Третьим объектом были сертификаты X.509, поскольку криптографические активы часто присутствуют не только в коде, но и в виде файлов, используемых приложением или тестовой инфраструктурой. Четвертым объектом была база знаний о квантово-уязвимых функциях, содержащая не только имя функции, но и ее категорию, алгоритм, библиотеку, оценку риска и возможные рекомендации по замене. Пятым объектом являлся итоговый отчет, объединяющий результаты всех этапов.

Такой способ декомпозиции позволил сформулировать требования не как общий список возможностей, а как набор требований к отдельным слоям системы. Это оказалось важно для реализации, поскольку впоследствии каждый слой был представлен отдельным модулем C++-кода.

Требования к инструменту были разделены на несколько групп:

3.1.1 Требования к входным данным и запуску инструмента

Инструмент должен запускаться из командной строки и принимать путь к анализируемому проекту. Командный интерфейс был выбран как основной, поскольку предполагаемый сценарий использования связан с анализом C/C++-кодовых баз, которые часто проверяются в терминале, на сервере сборки, в CI/CD-среде или в изолированной исследовательской среде. Графический интерфейс на этом этапе не требовался, так как он усложнил бы разработку, но не добавил бы существенной практической ценности.

Функционально инструмент должен поддерживать параметры, перечисленные в таблице 4.1.

Таблица 3.1 – Параметры и флаги командной строки

Флаг	Описание
-s, --source <path>	путь к исходному коду проекта
-c, --cert <path>	путь к файлу или директории сертификатов
-d, --db <path>	путь к JSON-базе уязвимых функций
-m, --mode <regex ast>	режим анализа кода
-I, --include <dir>	include-директория для AST-анализа
-o, --output <path>	путь к полному JSON-отчету
--cbom-output <path>	путь к CBOM-отчету
--extra-db <path>	дополнительная база сигнатур
--metrics	включение расчета precision/recall/F1
--ground-truth <path>	файл эталонной разметки
--cbom-only	генерация только CBOM
--skip-cert	пропуск анализа сертификатов
-v, --verbose	подробный режим вывода

Разделение параметров на обязательные и дополнительные позволяет использовать инструмент как в минимальном режиме, так и в расширенном: с сертификатами, метриками, путями к заголовочным файлам и дополнительными базами данных.

3.1.2 Требования к инвентаризации файлов проекта

FR-01. Инструмент должен принимать на вход путь к проекту и выполнять рекурсивный обход его файловой структуры. Обход должен учитывать реальные особенности C/C++-проектов: наличие исходных файлов, заголовочных файлов, файлов сборки, тестов, конфигураций и сертификатов. При этом служебные директории, не относящиеся к анализируемому исходному коду, должны исключаться из обработки, чтобы снизить шум и ускорить анализ.

FR-02. Для каждого обнаруженного файла инструмент должен определять категорию: исходный файл, заголовочный файл, тестовый файл, файл сборки, конфигурационный файл, документация, ресурс, сертификат или прочий файл. Такое разделение необходимо не только для отчета, но и для последующей оценки риска. Например, криптографический вызов в тестовом файле не должен получать такой же приоритет миграции, как вызов в сетевом модуле production-кода.

FR-03. Инструмент должен формировать структурированный объект инвентаризации проекта, включающий путь проекта, имя проекта, список файлов, размеры файлов, число строк, язык и статистику по категориям. Этот объект должен использоваться на следующих этапах: статическом анализе, оценке контекста риска и генерации отчета.

3.1.3 Требования к модулю статического анализа исходного кода

FR-04. Инструмент должен поддерживать режим анализа на основе регулярных выражений. Этот режим предназначен для быстрого первичного сканирования и должен работать без внешних зависимостей, кроме стандартной библиотеки C++. Он должен искать вызовы уязвимых функций по заранее скомпилированным паттернам, заданным в базе данных криптографических сигнатур.

FR-05. Инструмент должен поддерживать AST-режим анализа с использованием libclang. Этот режим должен извлекать вызовы функций из дерева разбора, определять имя вызываемой функции, строку и столбец, объемлющую функцию, класс, пространство имен и список аргументов.

FR-06. AST-режим должен выполнять не только поверхностный обход дерева, но и межпроцедурный анализ. Для C/C++ это существенно, поскольку

ку криптографический вызов часто находится внутри вспомогательной функции, а значение алгоритма передается через параметры, локальные переменные или поля класса. Поэтому анализатор должен строить сводки функций, граф вызовов, учитывать достижимость из корневых функций и пытаться разрешать значения аргументов.

FR-07. Инструмент должен предусматривать fallback-механизм на случай отсутствия `libclang` или невозможности построить AST из-за отсутствующих `include`-путей. В этом случае анализ не должен аварийно завершаться. Допустимо использование более простого токенового анализа, который удаляет комментарии и строковые литералы, разбивает код на токены и ищет последовательности определенного вида.

3.1.4 Требования к базе данных криптографических сигнатур

FR-08. База уязвимых функций должна храниться во внешнем JSON-файле. Это требование принципиально важно для расширяемости: добавление новых библиотек, алгоритмов, функций и рекомендаций по замене не должно требовать перекомпиляции инструмента.

FR-09. Для каждой уязвимой функции база должна содержать идентификатор, имя функции, алиасы, категорию, алгоритм, библиотеку, описание квантовой уязвимости, базовую оценку риска, ссылки на нормативные документы, регулярные выражения для поиска, контекстные ключевые слова и рекомендации по замене.

FR-10. База должна поддерживать несколько библиотек. В текущей реализации она включает `OpenSSL`, `Crypto++` и отдельный блок для ГОСТ-совместимых вызовов через `gost_openssl`. Такое решение отражает практическую задачу ВКР: анализировать не только международные криптографические API, но и российский нормативный контекст.

FR-11. Инструмент должен поддерживать подключение дополнительных JSON-баз через параметр командной строки. Это позволяет использовать базовую поставку для типовых библиотек и дополнять ее корпоративными или отраслевыми сигнатурами.

3.1.5 Требования к модулю анализа сертификатов

FR-12. Инструмент должен анализировать X.509-сертификаты, переданные отдельным файлом или директорией. Анализ сертификатов

необходим, поскольку квантово-уязвимая криптография присутствует не только в исходном коде, но и в инфраструктурных артефактах: TLS-сертификатах, цепочках доверия, локальных тестовых сертификатах и embedded-сертификатах.

FR-13. Для каждого сертификата инструмент должен извлекать subject, issuer, серийный номер, срок действия, алгоритм подписи, алгоритм открытого ключа, размер ключа, признак самоподписанности, признак истечения срока действия и оценку квантового риска. Структура X.509-сертификатов, включая поля субъекта, эмитента, открытого ключа и алгоритма подписи, определяется RFC 5280 (RFC 5280).

FR-14. Для разбора сертификатов должен использоваться OpenSSL API.

3.1.6 Требования к модулю оценки рисков и формирования плана миграции

FR-15. Каждая находка статического анализатора должна получать базовую оценку риска из базы сигнатур. Эта оценка отражает криптографическую природу алгоритма: RSA, DSA, ECDSA, ECDH и DH получают высокий риск, поскольку их безопасность основана на факторизации или дискретном логарифмировании, то есть на задачах, уязвимых к алгоритму Шора (Shor 1997).

FR-16. Итоговая оценка риска должна учитывать контекст использования. Один и тот же вызов RSA может иметь разный приоритет в зависимости от того, находится ли он в тестовом коде, сетевом модуле, модуле хранения данных или коде управления ключами. Поэтому базовая оценка должна модифицироваться контекстными множителями.

FR-17. Инструмент должен формировать приоритизированный план миграции. Элемент плана должен содержать место находки, текущую функцию, библиотеку, рекомендуемую функцию или технологию замены, стандарт, алгоритм, пояснение по миграции, ссылку на NIST и примечание по ТК 26.

FR-18. План миграции должен сортироваться по убыванию итоговой оценки риска. Это позволяет использовать отчет не только как инвентарь, но и как рабочий список задач для разработки.

3.1.7 Требования к отчетности

FR-19. Инструмент должен генерировать полный JSON-отчет, включающий сведения об инструменте, режиме анализа, исходном пути, инвентаризации проекта, находках, сертификатах, оценке риска, плане миграции, соответствии NIST и рекомендациях в контексте ТК 26.

FR-20. Инструмент должен формировать SBOM в формате CycloneDX 1.5. SBOM нужен для машиночитаемого представления криптографических активов, которое может быть передано в внешние системы управления рисками или комплаенсом (CycloneDX SBOM).

FR-21. Отчет должен содержать секцию NIST compliance, отражающую этапы перехода на постквантовую криптографию. Сроки и логика перехода должны согласовываться с NIST IR 8547, где описан переход от квантово-уязвимых алгоритмов на постквантовые стандарты.

3.1.8 Нефункциональные требования

NFR-01. Инструмент должен быть платформо-независимым и поддерживать систему сборки CMake. В текущей реализации поддерживаются Linux, macOS и Windows при наличии необходимых зависимостей.

NFR-02. Архитектура должна быть модульной. Инвентаризация, база сигнатур, анализ кода, анализ сертификатов, оценка риска, расчет метрик и генерация отчетов должны быть реализованы как отдельные компоненты.

NFR-03. Инструмент должен быть расширяемым. Расширяемость обеспечивается на двух уровнях: через подключение новых языковых плагинов и через подключение дополнительных баз уязвимостей.

NFR-04. Инструмент должен допускать частичный анализ даже при неполной конфигурации исследуемого проекта.

NFR-05. Отчет должен быть воспроизводимым и пригодным для дальнейшей автоматизированной обработки. Поэтому основным форматом результата выбран JSON, а криптографический инвентарь дополнительно экспортируется в SBOM.

3.2 Общая архитектура и процесс обработки

Архитектура инструмента построена по принципу конвейерной обработки. Каждый компонент решает отдельную задачу и передает результат

выполнения следующему модулю. Такой подход был выбран по нескольким причинам.

Во-первых, задачу криптографической инвентаризации интуитивно просто разбить на этапы: сначала нужно понять файловую структуру проекта, затем обнаружить криптографические вызовы, проанализировать сертификаты, оценить риск и только после этого сформировать отчет.

Во-вторых, модульная архитектура упрощает тестирование. Каждый компонент можно проверять независимо: базу сигнатур, файловый сканер, regex-анализатор, AST-анализатор, анализатор сертификатов, оценщик риска и генератор отчетов.

В-третьих, такой подход облегчает дальнейшее расширение. Например, можно добавить новый языковой плагин, не внося изменений в модуль оценки риска. Можно расширить список квантовых уязвимостей, не меняя структуру анализа. Можно добавить новый формат отчета, а сам алгоритм обнаружения оставить неизменным.

Общая архитектура инструмента изображена на рисунке ??.

```
const char *algo = get_config("cipher");
EVP_CIPHER_CTX *ctx = EVP_CipherInit_ex(NULL, EVP_get_cipherbyname(algo), ...);
```

Рисунок 3.1 – Поток обработки данных

Сам процесс обработки можно представить как последовательность вызовов (рисунок 5.1). Сначала main.cpp вызывает разбор конфигурации. Затем загружается база уязвимостей. Далее выполняется сканирование проекта. После этого выбирается режим анализа кода: regex или AST. Затем при необходимости анализируются сертификаты. Далее выполняется оценка рисков. Последним этапом создаются отчеты.

```
const char *algo = get_config("cipher");
EVP_CIPHER_CTX *ctx = EVP_CipherInit_ex(NULL, EVP_get_cipherbyname(algo), ...);
```

Рисунок 3.2 – Поток обработки данных

Внутренние структуры данных были спроектированы так, чтобы каждая из них соответствовала определенному уровню обработки. AppConfig описывает параметры запуска. ProjectInventory описывает структуру проекта. Finding описывает обнаруженный криптографический вызов. CertInfo описывает сертификат. RiskReport описывает результаты оценки риска.

MigrationAction описывает одно действие плана миграции. Cbom описывает человекочитаемый криптографический инвентарь.

AppConfig является конфигурационным объектом всего приложения. Он создается в самом начале работы из аргументов командной строки и далее передается тем модулям, которым нужны параметры запуска.

```
1 struct AppConfig {
2     std::string source_path;
3     std::string cert_path;
4     std::string db_path;
5     std::string output_path;
6     std::string cbom_output_path;
7     AnalysisMode analysis_mode;
8     bool verbose;
9     bool cbom_only;
10    bool skip_cert;
11    bool run_metrics;
12    std::string ground_truth_path;
13    std::vector<std::string> extra_db_paths;
14    std::vector<std::string> include_dirs;
15 };
```

Листинг 1 – Пример функции

Такое централизованное хранение параметров исключает ситуацию, когда отдельные модули самостоятельно читают аргументы командной строки. Это повышает связность конфигурации и делает поведение инструмента предсказуемым.

Особенно важна структура Finding, поскольку она является связующим звеном между анализаторами, оценщиком риска и генератором отчетов. Она должна быть достаточно общей, чтобы подходить и для regex-режима, и для AST-режима. При этом она должна позволять AST-режиму передавать более подробные данные: класс, пространство имен и аргументы.

Критически значимый фрагмент структуры Finding приведен в листинге 2.

```
1 struct Finding {
2     std::string file_path;
3     std::string function_name;
4     std::string library;
```

```

5      std::string algorithm;
6      std::string category;
7      std::string quantum_vulnerability;
8
9      std::string context_function;
10     std::string context_class;
11     std::string context_namespace;
12
13     std::string raw_line;
14     std::string analyzer_mode;
15     std::string vuln_id;
16     std::string nist_reference;
17     std::string tc26_reference;
18
19     int line_number;
20     int column;
21     double base_risk_score;
22
23     std::vector<std::string> arguments;
24 };

```

Листинг 2 – Пример функции

Этот фрагмент важен для понимания всей архитектуры. Поля *file_path*, *line_number* и *column* определяют место находки. Поля *function_name*, *library*, *algorithm* и *category* описывают криптографическую сущность. Поля *context_function*, *context_class* и *context_namespace* делают результат объяснимым для разработчика. Поле *base_risk_score* связывает находку с оценкой риска. Поле *arguments* необходимо для более точного *AST*-анализа, особенно для универсальных API OpenSSL.

При проектировании также была заложена возможность расширения. Расширяемость реализуется на трех уровнях. Первый уровень – база сигнатур: новые функции можно добавить в *JSON*. Второй уровень – языковые плагины: новые типы файлов можно добавить через реализацию *ILanguagePlugin*. Третий уровень – анализаторы: новый способ анализа можно добавить через реализацию интерфейса *ICodeAnalyzer*.

4 РЕАЛИЗАЦИЯ ИНСТРУМЕНТА

4.1 Общая организация разработки

Проект реализован на C++17 и организован по классической схеме с разделением заголовочных файлов, исходных файлов, данных и тестов. Заголовочные файлы находятся в каталоге `include/pqc`, а реализации классов расположены в каталоге `src`. Данные базы уязвимых функций вынесены в отдельный JSON-файл. Тестовые данные и тесты размещены в каталоге `tests`.

Общая структура проекта изображена на рисунке ??

Для сборки используется CMake версии не ниже 3.16. В проекте подключается библиотека `nlohmann/json`, `OpenSSL` и, при наличии, `libclang`. `OpenSSL` является обязательной зависимостью, поскольку без него невозможно выполнить анализ X.509-сертификатов. `Libclang` является опциональной зависимостью, так как режим регулярных выражений и токенный анализатор могут использоваться и без него.

Внутренний поток данных можно увидеть на рисунке ??

Важной особенностью реализации является то, что все ключевые результаты представлены как структуры данных, которые можно сериализовать в JSON. Это упрощает отладку, тестирование и последующую интеграцию инструмента в процессы непрерывного контроля безопасности.

4.2 Реализация конфигурации и командного интерфейса

Командный интерфейс реализован в модуле `config.cpp` и описан структурой `AppConfig`. Пользователь управляет работой инструмента через параметры запуска. Основным обязательным параметром является путь к анализируемому проекту. В таблице 4.1 описаны основные параметры запуска.

Таблица 4.1 – Параметры и флаги командной строки

Флаг	Описание
<code>-s, --source <path></code>	путь к исходному коду проекта
<code>-c, --cert <path></code>	путь к файлу или директории сертификатов
<code>-d, --db <path></code>	путь к JSON-базе уязвимых функций

Флаг	Описание
<code>-m, --mode <regex ast></code>	режим анализа кода
<code>-I, --include <dir></code>	include-директория для AST-анализа
<code>-o, --output <path></code>	путь к полному JSON-отчету
<code>--cbom-output <path></code>	путь к CBOM-отчету
<code>--extra-db <path></code>	дополнительная база сигнатур
<code>--metrics</code>	включение расчета precision/recall/F1
<code>--ground-truth <path></code>	файл эталонной разметки
<code>--cbom-only</code>	генерация только CBOM
<code>--skip-cert</code>	пропуск анализа сертификатов
<code>-v, --verbose</code>	подробный режим вывода

Фрагмент структуры конфигурации, приведенной в листинге 24 показывает, что все параметры приводятся к объекту одного класса.

```

1 struct AppConfig {
2     std::string source_path;
3     std::string cert_path;
4     std::string db_path;
5     std::string output_path;
6     std::string cbom_output_path;
7     AnalysisMode analysis_mode = AnalysisMode::REGEX;
8
9     bool verbose = false;
10    bool cbom_only = false;
11    bool skip_cert = false;
12    bool run_metrics = false;
13
14    std::string ground_truth_path;
15    std::vector<std::string> extra_db_paths;
16    std::vector<std::string> include_dirs;
17

```

```
18 static AppConfig parse(int argc, char** argv);  
19 };
```

Листинг 3 – Пример функции

Выбор режима анализа выполняется через перечисление `AnalysisMode`. Это исключает использование произвольных строковых значений во внутренних модулях и снижает вероятность ошибок. Строковые значения `regex` и `ast` используются только на границе программы, при разборе аргументов командной строки. После этого вся программа работает с типизированным значением.

Причина выделения конфигурации в отдельный модуль состоит в том, что конфигурация затрагивает практически все последующие этапы. Например, `db_path` нужен базе уязвимых функций, `analysis_mode` нужен выбору анализатора, `cert_path` и `skip_cert` нужны модулю сертификатов, а `ground_truth_path` нужен модулю метрик. Если бы разбор аргументов был распределен по разным частям программы, поддерживать согласованность стало бы намного сложнее.

4.3 Реализация базы уязвимых функций

База уязвимых функций является одним из центральных элементов инструмента. Она хранится в файле `data/vulnerable_functions.json` и загружается классом `VulnDatabase`. В текущей реализации база имеет версию 2.0 и включает функции из нескольких групп библиотек: `OpenSSL`, `Crypto++` и реализации ГОСТ-алгоритмов.

Содержательно база была сформирована как перечень криптографических API, использование которых указывает на применение алгоритмов, потенциально уязвимых к квантовым атакам или уже признанных устаревшими. В нее включены алгоритмы `RSA`, `DSA`, `DH`, `ECDH`, `ECDSA`, отдельные EVP-вызовы `OpenSSL`, низкоуровневые структуры ключей, а также устаревшие хэш-функции. Отдельно включены функции и обозначения для ГОСТ-алгоритмов, поскольку при разработке учитывалась не только международная, но и российская криптографическая практика.

Обобщенную структуру записи базы данных описана в листинге 4.

```
1 {  
2   "id": "openssl-rsa-keygen",
```

```

3  "name": "RSA_generate_key",
4  "library_name": "openssl",
5  "category": "asymmetric_encryption",
6  "algorithm": "RSA",
7  "quantum_vulnerability": "Shor algorithm",
8  "risk_score": 10.0,
9  "aliases": ["RSA_generate_key_ex"],
10 "patterns": ["\\bRSA_generate_key\\s*\\(", "\\bRSA_generate_key_ex\\s*\\(",
11             ],
12 "context_keywords": ["key", "certificate", "tls"],
13 "nist_reference": "NIST PQC migration recommendations",
14 "tc26_reference": "TC26 migration note",
15 "replacements": [
16 {
17     "function_name": "ML-KEM",
18     "library": "OpenSSL provider or PQC library",
19     "standard": "FIPS 203",
20     "algorithm": "ML-KEM",
21     "type": "key_encapsulation",
22     "migration_notes": "Replace RSA key transport with KEM-based or hybrid
23                       scheme",
24     "example_code": "...",
25     "tc26_note": "Consider national regulatory requirements"
26 }
27 ]
28 }

```

Листинг 4 – Пример функции

В реальном файле база содержит не одну запись, а набор записей для разных библиотек и алгоритмов. Группы функций, относящихся к OpenSSL, представлены в таблице 4.2.

Таблица 4.2 – Классификация функций криптографических библиотек для миграции

Группа	Примеры функций	Причина включения
RSA	RSA_generate_key, RSA_generate_key_ex, RSA_public_encrypt, RSA_private_decrypt, RSA_sign, RSA_verify	RSA уязвим к квантовой атаке на факторизацию.
DSA	DSA_sign, DSA_verify, DSA_do_sign, DSA_do_verify	DSA основан на сложности дискретного логарифмирования.
DH	DH_generate_key, DH_compute_key	Классический Диффи-Хеллман уязвим при наличии квантового компьютера.
ECDH	ECDH_compute_key	Эллиптический Диффи-Хеллман также относится к уязвимым схемам.
ECDSA	ECDSA_sign, ECDSA_verify, ECDSA_do_sign, ECDSA_do_verify	Подпись на эллиптических кривых уязвима к квантовой атаке.
EVP	EVP_PKEY_encrypt, EVP_PKEY_decrypt, EVP_DigestSignInit, EVP_PKEY_CTX_new	Высокоуровневые вызовы OpenSSL могут скрывать RSA, ECDSA, DH и другие алгоритмы.
BIGNUM	отдельные операции BN_*	Могут быть признаком ручной реализации криптографических примитивов.

Группа	Примеры функций	Причина включения
Устаревшие хэши	SHA1, MD5	Не являются основной целью постквантовой миграции, но важны для общей криптографической инвентаризации.

Классы и типы, характерные для библиотеки Crypto++, указаны в таблице 4.3.

Таблица 4.3 – Классификация объектов библиотеки Qrypto++ по уровням риска

Запись	Псевдонимы	Риск
RSA::PrivateKey	RSA::PublicKey, InvertibleRSAFunction	Высокий
RSAES_OAEP_SHA_Encryptor	RSAES_PKCS1v15_Encryptor, RSAES_OAEP_SHA_Decryptor	Высокий
ECDH	ECMQV	Высокий
ECDSA	ECGDSA, ECNR	Высокий
DH2	DH, MQV	Высокий

Для ГОСТ-направления включены записи gost_ec_sign, gost_ec_verify, GOST_R3410_keypair, VKO_cryptopro_2001, VKO_cryptopro_2012, gost_vko_compute. Их включение важно, поскольку инструмент должен быть применим не только к проектам, использующим OpenSSL, но и к коду, в котором встречаются национальные криптографические механизмы.

Класс VulnDatabase отвечает за загрузку базы, объединение основной и дополнительных баз, построение индекса имен и поиск функций.

```

1  class VulnDatabase {
2      public:
3      bool load(const std::string& path);

```

```

4      bool merge_json(const nlohmann::json& j);
5
6      const VulnerableFunction* find_by_name(
7      const std::string& name
8      ) const;
9
10     const std::vector<VulnerableFunction>& all() const;
11     std::vector<VulnerableFunction> by_library(
12     const std::string& library
13     ) const;
14
15     std::string db_version() const;
16     std::string last_updated() const;
17     std::size_t size() const;
18
19     private:
20     std::vector<VulnerableFunction> functions_;
21     std::unordered_map<std::string, std::size_t> name_index_;
22
23     void rebuild_index();
24     static std::string to_lower(std::string s);
25 };

```

Листинг 5 – Пример функции

Ключевой технический момент состоит в методе `rebuild_index`. После загрузки база строит индекс по имени функции и по всем псевдонимам. Поиск выполняется без учета регистра. Это позволяет анализатору не перебирать весь массив функций при каждой находке, а быстро получать описание уязвимой функции по имени.

Такое решение также позволяет подключать дополнительные базы через параметр `—extra-db`. Например, если организация использует собственную криптографическую обертку `CompanyRSAEncrypt`, ее можно добавить во внешний JSON-файл, не изменяя исходный код анализатора.

4.4 Реализация инвентаризации проекта

Инвентаризация проекта реализована классом `ProjectScanner`. Ее задача состоит в том, чтобы до запуска анализа получить полную картину состава проекта. Это необходимо по двум причинам. Во-первых, анализаторы долж-

ны получать только релевантные файлы. Во-вторых, контекст файла влияет на риск: тестовый код, сетевой код и код хранения данных должны интерпретироваться по-разному.

Для классификации файлов используется расширяемая система языковых плагинов, описанная в листинге 6.

```
1 class ILanguagePlugin {
2     public:
3     virtual ~ILanguagePlugin() = default;
4
5     virtual std::string language_name() const = 0;
6
7     virtual bool matches(
8     const std::filesystem::path& path
9     ) const = 0;
10
11     virtual FileCategory categorize(
12     const std::filesystem::path& path
13     ) const = 0;
14 };
```

Листинг 6 – Пример функции

В текущей реализации предусмотрены плагины для C/C++-файлов, CMake-файлов и сертификатов. Плагин C++ распознает исходные и заголовочные файлы по расширениям. Плагин CMake относит CMakeLists.txt и похожие файлы к сборочным. Плагин сертификатов определяет файлы сертификатов по расширениям, характерным для PEM, CRT, CER и DER. Фрагмент логики сканирования описан в листинге 7.

```
1 ProjectInventory ProjectScanner::scan(
2     const std::string& root_path
3 ) const {
4     ProjectInventory inventory;
5     inventory.project_path = root_path;
6     inventory.project_name =
7     std::filesystem::path(root_path).filename().string();
8
9     for (const auto& entry :
10     std::filesystem::recursive_directory_iterator(root_path)) {
11         if (!entry.is_regular_file()) {
```

```

12         continue;
13     }
14
15     if (should_skip(entry.path())) {
16         continue;
17     }
18
19     FileEntry file;
20     file.path = entry.path().string();
21     file.size_bytes = entry.file_size();
22     file.category = detect_category(entry.path());
23     file.language = detect_language(entry.path());
24     file.line_count = count_lines(entry.path());
25
26     inventory.files.push_back(file);
27     inventory.language_stats[file.language]++;
28 }
29
30 return inventory;
31 }

```

Листинг 7 – Пример функции

В реальном коде сканер также исключает служебные каталоги. Это важно, потому что анализ каталогов сборки, зависимостей или истории Git увеличил бы число ложных срабатываний и замедлил бы работу. Например, каталог `build` может содержать сгенерированные файлы, а `.git` содержит служебные объекты, которые не должны рассматриваться как исходный код проекта.

Результатом сканирования является `ProjectInventory`. Он может быть выведен в виде сводки, использован для выбора файлов анализа и преобразован в метаданные СВМ. Например, если проект содержит 200 файлов, но только 35 из них относятся к исходному коду C++, анализаторы получают именно эти 35 файлов, а не документацию, изображения или служебные ресурсы.

На рисунке ?? показана внутренняя логика инвентаризации.

Главное преимущество такой схемы заключается в том, что добавление нового типа файлов не требует переписывания сканера. Достаточно реализовать новый плагин, который сможет проанализировать файл и определить его

категорию.

4.5 Реализация общего интерфейса анализаторов

Общий интерфейс анализаторов введен для того, чтобы разные методы статического анализа могли быть взаимозаменяемыми. В проекте реализованы три подхода: анализ регулярными выражениями, анализ на уровне токенов и AST. Объявление самого интерфейса описано в листинге ??.

```
1 class ICodeAnalyzer {
2     public:
3     virtual ~ICodeAnalyzer() = default;
4
5     virtual std::string mode_name() const = 0;
6
7     virtual std::vector<Finding> analyze(
8         const ProjectInventory& inventory,
9         const VulnDatabase& db
10    ) = 0;
11
12    virtual std::vector<Finding> analyze_file(
13        const FileEntry& file,
14        const VulnDatabase& db
15    ) = 0;
16 };
```

Листинг 8 – Пример функции

Метод `analyze` работает с полной инвентаризацией проекта, а `analyze_file` анализирует один файл. Такое разделение удобно для тестирования: модульные тесты могут проверять отдельный файл, а интеграционные тесты могут проверять полный конвейер.

Единый интерфейс также позволяет сравнивать разные режимы анализа на одинаковых тестовых данных. Например, один и тот же файл `sample_vulnerable.cpp` может быть передан `RegexAnalyzer` и `ASTAnalyzer`, после чего результаты сравниваются с эталонной разметкой.

Важно отметить, что все результаты имеют общий формат, который в дальнейшем передается на вход модулю оценки риска. С точки зрения архитектуры это решение является принципиально важным: если в будущем будет добавлен анализатор для Python, Java или Go, он также сможет возвра-

щать Finding, а остальные этапы останутся без изменений.

4.6 Реализация анализатора регулярных выражений

Анализатор регулярных выражений реализован в классе RegexAnalyzer. Он является самым простым и быстрым из режимов анализа. Его задача: построчно прочитать исходный файл и найти в нем вызовы функций, описанных в базе уязвимых функций.

Для каждой функции из базы используется два источника шаблонов: явно заданные регулярные выражения из поля patterns и автоматически сформированный шаблон. Это позволяет находить вызовы, даже если для функции не задан отдельный шаблон. Принцип его работы представлен в листинге 9.

```
1 for (const auto& vf : db.all()) {
2     std::vector<std::regex> compiled_patterns;
3
4     for (const auto& pattern : vf.patterns) {
5         compiled_patterns.emplace_back(pattern);
6     }
7
8     std::string auto_pattern =
9     "\\b" + escape_regex(vf.name) + "\\s*\\(";
10    compiled_patterns.emplace_back(auto_pattern);
11
12    for (const auto& re : compiled_patterns) {
13        auto begin = std::sregex_iterator(
14            line.begin(), line.end(), re
15        );
16        auto end = std::sregex_iterator();
17
18        for (auto it = begin; it != end; ++it) {
19            Finding finding;
20            finding.file_path = file.path;
21            finding.function_name = vf.name;
22            finding.library = vf.library_name;
23            finding.algorithm = vf.algorithm;
24            finding.category = vf.category;
25            finding.quantum_vulnerability =
26            vf.quantum_vulnerability;
27            finding.line_number = line_number;
```

```

28         finding.column = static_cast<unsigned>(
29             it->position() + 1
30         );
31         finding.raw_line = line;
32         finding.analyzer_mode = mode_name();
33         finding.vuln_id = vf.id;
34         finding.base_risk_score = vf.risk_score;
35
36         out.push_back(finding);
37     }
38 }
39 }

```

Листинг 9 – Пример функции

На практике в реализации также используется дедупликация по имени функции и номеру строки. Это необходимо, потому что одна и та же функция может быть найдена одновременно по явному шаблону и по автоматически построенному шаблону. Без дедупликации отчет содержал бы повторяющиеся находки, что исказило бы оценку риска и расчет нужных метрик.

У анализатора регулярных выражений есть важное достоинство: он быстро работает и не требует внешних зависимостей, кроме стандартной библиотеки C++. Его можно использовать для первичной оценки проекта и для окружений, где недоступен libclang.

Однако у этого режима есть и ограничения. Регулярное выражение анализирует текст, а не структуру программы. Поэтому оно может сработать на строковом литерале, многострочном комментарии или макросе. В текущей реализации частично учитываются однострочные комментарии, но полноценное понимание синтаксиса C++ в этом режиме не выполняется. Именно для борьбы с этой проблемой и был реализован анализатор на уровне AST.

4.7 Реализация AST-анализатора

AST-анализатор является наиболее сложным компонентом инструмента. Он реализован в классе `ASTAnalyzer` и использует `libclang` для построения абстрактного синтаксического дерева C++-кода. В отличие от регулярных выражений и токенов, AST-анализатор работает не только с текстом, но и со структурой программы: функциями, методами, классами, вызовами, ар-

гументами, локальными переменными и областями видимости.

4.7.1 Парсинг единицы трансляции

Для каждого анализируемого исходного файла создается своя единица трансляции. Помимо стандартных параметров в `libclang` также передаются пути к `include`-директориям, указанным пользователем.

Концептуально этот этап выглядит так (листинг 10):

```
1 const char* args[] = {  
2     "-x", "c++",  
3     "-std=c++17",  
4     "-w",  
5     "-ferror-limit=0"  
6 };  
7  
8 CXTranslationUnit tu = clang_parseTranslationUnit(  
9     index,  
10    file.path.c_str(),  
11    args,  
12    arg_count,  
13    nullptr,  
14    0,  
15    CXTranslationUnit_DetailedPreprocessingRecord |  
16    CXTranslationUnit_KeepGoing  
17 );
```

Листинг 10 – Пример функции

Последний флаг (игнорирование некритичных ошибок) очень важен для практического анализа. В реальных проектах отдельный файл может не разбираться идеально из-за отсутствующих зависимостей или специфических флагов сборки. Задача инструмента не состоит в полной компиляции проекта, поэтому анализатор должен по возможности продолжать работу даже при наличии ошибок разбора.

4.7.2 jjj

Первая фаза `ASTAnalyzer` собирает сведения обо всех определенных функциях, методах, конструкторах и деструкторах. Для каждой функции создается свой `FunctionSummary` (листинг ??):

```

1 struct FunctionSummary {
2     std::string qualified_name;
3     std::string simple_name;
4     std::string file_path;
5
6     CXCursor cursor{};
7     bool has_cursor = false;
8
9     bool is_static = false;
10    bool in_anon_ns = false;
11    bool is_definition = false;
12
13    std::vector<std::string> param_names;
14    std::vector<std::string> param_types;
15
16    std::unordered_map<std::string, std::vector<LocalValue>> locals;
17
18    CXCursor return_cursor{};
19    bool has_return_cursor = false;
20
21    std::vector<std::string> callees;
22 };

```

Листинг 11 – Пример функции

Эта структура показывает, что анализатор строит не просто список вызовов, а внутреннюю модель функций. `qualified_name` нужен для различения функций в разных классах и пространствах имен; `simple_name` нужен для сопоставлений по имени; `param_names` и `param_types` нужны для разрешения аргументов; `locals` хранит историю локальных переменных; `callees` используется для построения графа вызовов.

4.7.3 Сбор локальных значений

На второй фазе анализатор заполняет тела функций. Одной из ключевых задач является сбор значений локальных переменных. Для этого используется структура `LocalValue` (листинг 12):

```

1 struct LocalValue {
2     std::string text;
3     unsigned line = 0;

```

```

4   bool known = false;
5 };

```

Листинг 12 – Пример функции

Значения хранятся как история, а не как одно последнее значение. Это позволяет учитывать переопределения переменной. Если переменная имеет несколько присваиваний, при разрешении аргумента выбирается то значение, которое было актуально до строки вызова.

Пример ситуации, для которой нужен такой механизм, приведен в листинге ??.

```

1   const char* alg = "RSA";
2   EVP_PKEY_CTX* ctx = EVP_PKEY_CTX_new_from_name(nullptr, alg, nullptr);

```

Листинг 13 – Пример функции

Анализатор на основе регулярных выражений в такой ситуации увидит только вызов `EVP_PKEY_CTX_new_from_name`. AST-анализатор может попытаться понять, что аргумент `alg` соответствует строке "RSA". Это особенно важно для универсальных EVP-функций OpenSSL, где реальный алгоритм передается параметром.

4.7.4 Сбор полей классов и побочных эффектов

Особенность C++-кода состоит в том, что криптографические объекты часто хранятся в полях классов, а операции с ними выполняются в разных методах. Поэтому в AST-анализаторе реализованы дополнительные фазы сбора полей классов и эффектов методов.

Сведения о полях позволяют анализатору понимать, что объект, созданный или записанный в одном методе, может использоваться в другом. Для этого также используется структура событий метода (листинг ??).

```

1 struct FieldInfo {
2     std::string type_spelling;
3     std::string resolved_value;
4     bool has_value = false;
5 };
6
7 struct MethodEvent {
8     enum class Kind {

```

```

9      Mutation,
10     IntraClassCall
11 };
12
13 Kind kind;
14 unsigned line = 0;
15 std::string mutation;
16 std::string call_qualified;
17 };

```

Листинг 14 – Пример функции

Событие `Mutation` означает изменение состояния объекта, а `IntraClassCall` означает вызов другого метода того же класса. Это нужно для восстановления точной последовательности действий внутри объектно-ориентированного кода.

4.7.5 Построение достижимости

После сбора функций и их вызовов `ASTAnalyzer` вычисляет достижимость. Корнями считаются `main` и функции с внешней линковкой, то есть функции, которые потенциально могут вызываться извне `translation unit`. Далее выполняется BFS по графу вызовов.

Недостижимые функции не исключаются из отчета полностью. Вместо этого для них применяется понижающий коэффициент риска (листинг 15).

```

1 static constexpr double kUnreachableRiskFactor = 0.3;
2
3 f.base_risk_score = pc.reachable_from_root
4 ? opt->risk_score
5 : opt->risk_score * kUnreachableRiskFactor;

```

Листинг 15 – Пример функции

Это важное проектное решение. Если полностью исключить недостижимые функции, можно пропустить реальный риск, потому что анализ выполняется в пределах одной `translation unit` и может не видеть внешние точки входа. Если же не учитывать достижимость вообще, то мертвый или вспомогательный код будет получать такой же приоритет, как реально исполняемый. Использование коэффициента 0.3 является компромиссом: находка сохраняется, но ее приоритет заметно снижается.

4.7.6 Разрешение аргументов

Одна из наиболее важных возможностей AST-анализатора – разрешение аргументов вызова. Для этого используется ResolutionContext (листинг 16), который объединяет текущий фрейм функции, сводки функций, привязки параметров, реестр полей класса и динамическое состояние полей.

```
1 struct ResolutionContext {
2     FunctionFrame* frame;
3
4     const std::unordered_map<std::string, FunctionSummary*> summaries;
5
6     std::unordered_map<std::string, std::string> param_bindings;
7
8     std::unordered_set<std::string> active_functions;
9
10    const ClassFieldRegistry* field_registry;
11
12    std::string current_class_qualified;
13    std::string current_class_simple;
14
15    const FieldStateMap* dynamic_field_state;
16};
```

Листинг 16 – Пример функции

Схема процесса разрешения аргумента представлена на рисунке ??.

Такой механизм особенно важен для вызовов вида EVP_PKEY_CTX_new_from_name. В этом случае сама функция является универсальной, а конкретный алгоритм определяется строковым аргументом. Если ASTAnalyzer может разрешить этот аргумент, отчет становится существенно точнее.

4.7.7 Ограничения AST-анализатора

Несмотря на большую глубину анализа, AST-анализатор имеет значительные ограничения. Он анализирует единицы трансляции отдельно и не выполняет полноценный межмодульный анализ. Заголовочные файлы не анализируются как самостоятельные файлы с исходным кодом, хотя могут попадать в анализ через механизм включения. Вызовы через указатели на функции не разрешаются полностью. Глубина разрешения выражений, под-

становки функций и обхода графа вызовов ограничена константами (листинг 17):

```
1 static constexpr int kMaxResolveDepth = 12;
2 static constexpr int kMaxInlineExpansionDepth = 6;
3 static constexpr int kMaxCallGraphDepth = 8;
4 static constexpr std::size_t kMaxResolvedText = 512;
```

Листинг 17 – Пример функции

Эти ограничения необходимы для предотвращения бесконечной рекурсии и чрезмерного роста времени анализа. Они также задают границы применимости инструмента: AST-анализатор повышает точность по сравнению с регулярными выражениями, но не является полноценным компиляторным анализатором всех возможных путей выполнения.

4.8 Реализация анализа сертификатов

Модуль CertAnalyzer анализирует X.509-сертификаты как отдельный тип криптографических активов. Это важно, поскольку проект может содержать сертификаты, даже если соответствующие криптографические операции в исходном коде не отражены явно.

Алгоритм анализа сертификата состоит из нескольких этапов (рисунок ??). Сначала файл открывается и анализатор пытается прочитать его как PEM. Если PEM-чтение не удалось, выполняется попытка чтения DER. Такой порядок выбран потому, что PEM часто используется в репозиториях и может содержать цепочку сертификатов, а DER представляет бинарный формат одного сертификата.

Ключевые операции извлечения данных выполняются через OpenSSL API (листинг 18):

```
1 X509_NAME_print_ex(...);
2 ASN1_INTEGER_to_BN(...);
3 BN_bn2hex(...);
4 ASN1_TIME_print(...);
5 ASN1_TIME_diff(...);
6 X509_get0_signature(...);
7 X509_get0_pubkey(...);
8 EVP_PKEY_base_id(...);
9 EVP_PKEY_bits(...);
```

Листинг 18 – Пример функции

Оценка квантового риска сертификата выполняется по алгоритму открытого ключа и алгоритму подписи. RSA, ECDSA, ECDH, DSA, EC, Ed25519, Ed448 и ГОСТ классифицируются как высокий риск в рамках постквантовой модели. MD5 также получает высокий риск, но по причине классической криптографической слабости. SHA-1 получает средний риск. SHA-384 и SHA-512 получают низкий риск.

Формула численной оценки сертификата изображена на рисунке ??.

Уменьшение риска для просроченного сертификата связано с тем, что такой сертификат, как правило, уже не должен использоваться в активной инфраструктуре. Однако он все равно сохраняется в отчете, потому что может быть частью тестовой среды или legacy-конфигурации.

Ограничение модуля состоит в том, что он не проверяет цепочку доверия и не выполняет полную TLS-валидацию. Его задача - инвентаризация и классификация сертификатов как криптографических активов, а не проверка корректности PKI.

4.9 Реализация оценки рисков

Оценка риска реализована в классе RiskAssessor. Он получает на вход список находок, инвентаризацию проекта и список сертификатов. Результатом является RiskReport, содержащий общий риск, распределение по приоритетам, файловые сводки и план миграции.

Главная идея модуля: базовая опасность алгоритма должна корректироваться контекстом. Например, RSA_sign в тестовом файле и RSA_sign в модуле подписи производственных данных не должны иметь одинаковый итоговый приоритет. Контекст описывается структурой ContextFactors (листинг ??).

```
1 struct ContextFactors {  
2     bool is_network_facing = false;  
3     bool is_persistent_data = false;  
4     bool is_key_material = false;  
5     bool is_in_loop = false;  
6     bool is_test_code = false;  
7 }
```

```

8     int call_frequency = 1;
9     std::string data_classification;
10 };

```

Листинг 19 – Пример функции

В контексте данной реализации используются следующие множители (таблица 4.4):

Таблица 4.4 – Коэффициенты корректировки риска по факторам кода

Фактор	Множитель	Интерпретация
Тестовый код	0.40	Риск снижается, так как код не относится к рабочему контуру.
Сетевой модуль	1.30	Риск повышается из-за возможного внешнего воздействия.
Постоянно хранимые данные	1.25	Риск повышается из-за долговременной ценности данных.
Ключевой материал	1.20	Риск повышается при работе с ключами.
Вызов в цикле	1.10	Риск повышается из-за частоты использования.
Частота вызова больше 5	1.10	Риск повышается при многократном использовании функции.

Итоговая формула риска описана в листинге 20.

```

1 final_score = min(10.0, base_score * context_multiplier)

```

Листинг 20 – Формула риска

Фрагмент самой логики расчета представляется так (листинг ??):

```

1 RiskScore RiskAssessor::compute_risk(
2     const Finding& finding,
3     const ContextFactors& factors
4 ) const {
5     double multiplier = 1.0;

```

```

6
7     if (factors.is_test_code) {
8         multiplier *= 0.40;
9     }
10    if (factors.is_network_facing) {
11        multiplier *= 1.30;
12    }
13    if (factors.is_persistent_data) {
14        multiplier *= 1.25;
15    }
16    if (factors.is_key_material) {
17        multiplier *= 1.20;
18    }
19    if (factors.is_in_loop) {
20        multiplier *= 1.10;
21    }
22    if (factors.call_frequency > 5) {
23        multiplier *= 1.10;
24    }
25
26    RiskScore score;
27    score.base_score = finding.base_risk_score;
28    score.context_multiplier = multiplier;
29    score.final_score =
30    std::min(10.0, score.base_score * multiplier);
31    score.priority = priority_for(score.final_score);
32    score.factors = factors;
33
34    return score;
35 }

```

Листинг 21 – Пример функции

Кроме отдельных находок, RiskAssessor формирует агрегированные данные по файлам. Для каждого файла сохраняются количество находок, максимальный риск, средний риск, общий приоритет и список имен функций. Это помогает понять, какие файлы являются наиболее проблемными с точки зрения предстоящей миграции.

Общий риск проекта рассчитывается как среднее значение по наиболее значимым находкам, а затем корректируется с учетом максимального риска сертификатов. В текущей реализации сертификаты влияют на итоговую

оценку с весом 0.3, а находки в коде с весом 0.7 (листинг 22).

```
1 final_score = min(10.0, base_score * context_multiplier)
```

Листинг 22 – Формула риска

Также модуль определяет статус готовности к миграции. Если находок и сертификатов нет, проект считается соответствующим. Если есть критические находки или более трех высоких, статус определяется как not-started. При наличии отдельных высоких или большого числа средних находок статус определяется как in-progress. В остальных случаях используется near-compliant.

4.10 Реализация плана миграции

План миграции формируется на основе результатов оценки риска. Для каждой находки создается объект MigrationAction (листинг 23). Он содержит информацию о текущей функции, предлагаемой замене, стандарте, примере кода, примечании ТК 26 и рассчитанном риске.

```
1 struct MigrationAction {
2     std::string finding_id;
3     std::string file_path;
4     unsigned line_number = 0;
5     int migration_order = 0;
6
7     std::string current_function;
8     std::string current_library;
9
10    std::string replacement_function;
11    std::string replacement_library;
12    std::string replacement_standard;
13    std::string replacement_algorithm;
14    std::string replacement_type;
15    std::string migration_notes;
16    std::string example_code;
17
18    std::string nist_reference;
19    std::string tc26_note;
20
21    RiskScore risk;
22    MigrationPriority priority;
```

Листинг 23 – Пример функции

В текущей реализации для формирования действия используется первая рекомендация из списка replacements. Это является осознанным упрощением: база уже может хранить несколько вариантов миграции, но отчет выбирает первый как основной. В дальнейшем это можно расширить до выбора альтернатив в зависимости от политики организации, поддерживаемых библиотек или требований совместимости.

Пример записи плана миграции в отчете может выглядеть так (листинг ??):

```

1 {
2     "migration_order": 1,
3     "file_path": "src/tls.cpp",
4     "line_number": 42,
5     "current_function": "RSA_generate_key_ex",
6     "current_library": "openssl",
7     "replacement_function": "ML-KEM",
8     "replacement_standard": "FIPS 203",
9     "replacement_algorithm": "ML-KEM",
10    "replacement_type": "key_encapsulation",
11    "priority": "HIGH",
12    "migration_notes": "Replace RSA key transport with KEM-based or hybrid
                        scheme"
13 }
```

Листинг 24 – Пример функции

План миграции не выполняет автоматическое изменение исходного кода. Его задача на текущем этапе: дать разработчику и специалисту по безопасности обоснованный порядок работ. Это важно, потому что автоматическая замена криптографических механизмов требует глубокого понимания протокола, формата данных, совместимости и требований к ключевой инфраструктуре.

4.11 Реализация SBOM и генератора отчетов

Генерация отчетов реализована в ReportGenerator. Он формирует два основных результата: полный JSON-отчет и SBOM в формате CycloneDX 1.5.

Полный отчет ориентирован на человека, а СВМ ориентирован на машинную обработку.

Структура СВМ описана классами Cbom, CbomComponent и CbomAlgorithmProperties (листинг 25).

```
1 struct CbomAlgorithmProperties {
2     std::string primitive;
3     std::string executionEnvironment;
4     std::string implementationPlatform;
5     std::optional<std::string> parameterSetIdentifier;
6
7     std::vector<std::string> cryptoFunctions;
8
9     int classicalSecurityLevel = 0;
10    int quantumSecurityLevel = 0;
11
12    nlohmann::json to_json() const;
13 };
14
15 struct CbomComponent {
16     std::string bom_ref;
17     std::string type = "cryptographic-asset";
18     std::string name;
19     CbomAssetType assetType;
20
21     CbomAlgorithmProperties algorithmProperties;
22
23     std::string file_path;
24     std::string context_function;
25     std::string library_source;
26     std::string quantum_vulnerability;
27
28     unsigned line_number = 0;
29     double risk_score = 0.0;
30
31     nlohmann::json to_json() const;
32 };
33 struct Cbom {
34     std::string bomFormat="CycloneDX", specVersion="1.5", serialNumber,
35         timestamp;
36     int version=1;
```

```

36     std::vector<CbomComponent> components;
37     void add_component(CbomComponent c);
38     nlohmann::json to_json() const;
39     static Cbom create_new();
40 };

```

Листинг 25 – Пример функции

Для каждой находки создается компонент CBOM. В нем указывается имя алгоритма, путь к файлу, строка, библиотека, уровень риска и свойства алгоритма. Например, RSA получает примитив *asymmetric-encryption*, классический уровень стойкости, условный квантовый уровень и указание на функцию использования.

В реализации есть функции сопоставления. Например, *primitive_for()* преобразует внутреннее имя алгоритма в тип криптографического примитива. RSA, ECDH, ECDSA, DH, DSA, SHA-1, SHA-256, MD5 и ГОСТ-механизмы приводятся к соответствующим категориям. Для классического уровня стойкости используются приближенные значения: RSA соответствует 112 битам, эллиптические алгоритмы 128 битам, SHA-256 также 128 битам, SHA-1 80 битам, MD5 64 битам. Для квантового уровня в текущей реализации применяются условные значения: высокий риск дает 0, средний 50, низкий 128. Формат самого отчета отражен в таблице 4.5.

Таблица 4.5 – Структура разделов отчета анализа безопасности

Раздел отчета	Содержимое
Метаданные проекта	Имя проекта, путь, статистика файлов.
Находки	Все найденные вызовы уязвимых функций.
Сертификаты	Сведения о сертификатах и их рисках.
Оценка риска	Общий риск, распределение по приоритетам.
План миграции	Упорядоченные действия по замене.
Сводки по файлам	Количество находок и риск по каждому файлу.
Раздел NIST	Состояние готовности к миграции.

Раздел ТК 26	Примечания и соответствующие рекомендации.
CBOM	Криптографическая ведомость в отдельном JSON-файле.

Код генерации отчета представлен в листинге 26.

```

1 nlohmann::json ReportGenerator::build_full_report(
2  const ProjectInventory& inventory,
3  const std::vector<Finding>& findings,
4  const std::vector<CertInfo>& certs,
5  const RiskReport& risk_report
6 ) const {
7     nlohmann::json report;
8
9     report["generated_at"] = now_iso8601();
10    report["project"] = inventory.to_cbom_metadata();
11    report["findings"] = findings_to_json(findings);
12    report["certificates"] = certs_to_json(certs);
13    report["risk_report"] = risk_report.to_json();
14    report["nist_compliance"] =
15    nist_compliance_section(risk_report);
16    report["tc26_compliance"] =
17    tc26_compliance_section(risk_report);
18
19    return report;
20 }
```

Листинг 26 – Пример функции

Отдельное формирование СВОМ важно потому, что криптографическая ведомость может использоваться независимо от полного отчета. Например, ее можно загрузить в систему управления составом программного обеспечения, хранить как артефакт сборки или сравнивать между версиями проекта.

5 ТЕСТИРОВАНИЕ И ОЦЕНКА

5.1 Общий подход к тестированию

Инструмент реализован как конвейер из шести взаимозависимых модулей, каждый из которых находится на критическом пути итогового отчёта. Их взаимосвязь показана на рисунке 5.1.



Рисунок 5.1 – Конвейер обработки данных

Поскольку ошибка в любом из звеньев цепи может проявиться только на выходе, применяются два уровня тестирования. *Модульные тесты* (41 тест, 6 наборов) проверяют каждый компонент изолированно — это позволяет немедленно локализовать источник ошибки. *Интеграционные тесты* (6 тестов) запускают полный конвейер на реальном тестовом файле и сверяют итоговый план миграции с эталонной разметкой.

Все тесты реализованы на C++ с использованием собственного фреймворка *TestSuite/TestRunner*, не требующего сторонних зависимостей. Итого выполняется *47 тестов*, все проходят успешно. Классификация приведена в таблице 5.1.

Таблица 5.1 – Классификация тестов системы

Набор тестов (кол-во)	Что проверяется	Ключевые тест-кейсы
<i>VulnDatabase</i> (9)	Загрузка JSON, поиск по имени и псевдониму, слияние с дополнительной базой, наличие ссылок на FIPS и TK 26	<i>find_alias, replacements_present, merge_extra_db</i>
<i>ProjectScanner</i> (5)	Обход файловой системы, классификация по категориям, статистика языков, метаданные CBOM, плагин	<i>cpp_files_detected, cbom_metadata_json, custom_plugin</i>
<i>RegexAnalyzer</i> (8)	Обнаружение функций, атрибуты находки (строка, библиотека, ссылка NIST), отсутствие срабатываний на AES	<i>finds_rsa_in_fixture, no_false_positives_for_safe_code, finding_has_nist_reference</i>
<i>ASTAnalyzer</i> (7)	Обнаружение уязвимостей, извлечение контекста функции и класса, исключение системных заголовков	<i>context_function_extracted, context_class_extracted, no_system_header_findings</i>

Набор тестов (кол-во)	Что проверяется	Ключевые тест-кейсы
<i>RiskAssessor</i> (7)	Влияние контекста на итоговую оценку, сортировка плана миграции, наличие рекомендаций и ссылок на стандарты	<i>network_file_increases_risk</i> , <i>test_file_reduces_risk</i> , <i>replacement_info_present</i>
<i>MetricsCalculator</i> (5)	Вычисление TP, FP, FN с допуском строки; precision, recall, F1; сериализация в JSON	<i>perfect_match</i> , <i>line_tolerance</i> , <i>json_serialization</i>
<i>FullPipeline</i> (6)	Полный конвейер в режимах regex и AST; precision/recall $\geq 50\%$; наличие ссылок ТК 26 и FIPS; сериализация отчёта	<i>regex_pipeline_end_to_end</i> , <i>regex_precision_recall_gt50pct</i> , <i>migration_plan_has_fips_references</i>

5.2 Тестовые данные

Основным тестовым файлом является *tests/fixtures/sample_vulnerable.cpp*. Он организован в виде восьми тематических секций (A–H), каждая из которых моделирует отдельный реальный сценарий применения криптографии, и завершающего блока *SAFE* — фрагментов, которые *не должны* давать срабатываний. Такое разделение позволяет измерять precision и recall раздельно по категориям уязвимостей.

Структура тестового файла представлена в таблице 5.2.

Таблица 5.2 – Секции тестового файла *sample_vulnerable.cpp*

Секция	Класс-контейнер	Описание и ожидаемые находки
A. <i>tls</i>	<i>crypto::tls:: TLSHandshake</i>	RSA-ключ сервера: <i>RSA_new</i> , <i>RSA_generate_key_ex</i> ; EC-ключ и ECDH: <i>EC_KEY_generate_key</i> , <i>ECDH_compute_key</i> ; под- пись ECDSA
B. <i>storage</i>	<i>crypto::storage:: SecureStorage</i>	RSA-шифрование: <i>RSA_public_encrypt</i> ; под- пись RSA; хеширование SHA-1
C. DH	функция <i>setup_dh</i>	Параметры Диффи-Хеллмана: <i>DH_new</i> , <i>DH_generate_ parameters_ex</i>
D. MD5	функция <i>md5sum</i>	Устаревшая хеш-функция: <i>MD5</i>
E. <i>evp</i>	<i>crypto::evp:: EvpOperations</i>	EVP-API высокого уровня: <i>EVP_ PKEY_CTX_new_from_name</i> с классическим аргументом "RSA"; <i>EVP_PKEY_CTX_new</i> ; полная цепочка подпи- си и верификации <i>EVP_ DigestSign*/EVP_DigestVerify*</i> ; ECDH-согласование через <i>EVP_PKEY_derive*</i>
F. <i>dsa</i>	<i>crypto::dsa_legacy:: DsaOps</i>	Явные DSA-операции: <i>DSA_new</i> , <i>DSA_sign</i> ; проверяет, что ана- лизатор не смешивает DSA и ECDSA

Продолжение таблицы 5.2

Секция	Класс-контейнер	Описание и ожидаемые находки
G. <i>ec_ops</i>	<i>crypto::ec_ops::ECOperations</i>	Низкоуровневые EC-операции: <i>EC_GROUP_new_by_curve_name</i> , <i>EC_KEY_new</i> , <i>EC_KEY_generate_key</i>
H. <i>bn_ops</i>	<i>crypto::bn_ops::BNOperations</i>	BN-операции как индикатор ручной реализации RSA/DH: <i>BN_generate_prime_ex</i> , <i>BN_mod_exp</i>
SAFE	функции <i>safe_*</i>	AES-256-GCM через <i>EVP_EncryptInit_ex</i> : симметричный алгоритм, отсутствует в базе уязвимых функций. <i>EVP_PKEY_CTX_new_from_name</i> с аргументом "ML-KEM-768": постквантовый алгоритм, подавляется AST-анализатором через отслеживание провенанса аргумента

Полная эталонная разметка из 32 записей хранится в файле *tests/fixtures/ground_truth.json*. Каждая запись включает: имя ожидаемой функции, номер строки и *line_tolerance* (3–5 строк) — допуск на расхождение номеров строк между AST и regex-режимами. Перечень всех ожидаемых находок приведён в приложении ??.

Листинг 27 показывает типичную структуру записи в *ground_truth.json*.

```

1 {
2   "function_name": "EVP_PKEY_CTX_new_from_name",
3   "file_path": "tests / fixtures / sample_vulnerable.cpp",
4   "line_number": 121,
5   "line_tolerance": 3,
6   "notes": "Conditional: 'RSA' arg makes it vulnerable; 'ML-KEM
```

```
    -768' must NOT trigger"  
7 }
```

Листинг 27 – Структура записи эталонной разметки

Поле *notes* носит документальный характер и не используется в расчёте метрик. Допуск *line_tolerance* необходим потому, что AST-анализатор привязывает находку к точной строке вызова, тогда как regex-анализатор может зафиксировать строку начала многострочного выражения.

5.3 Модульные тесты

5.3.1 База данных уязвимых функций (*VulnDatabase*)

Назначение и структура. Класс *VulnDatabase* загружает JSON-файл *data/vulnerable_functions.json*, строит инвертированный индекс по именам и псевдонимам и предоставляет метод *find_by_name(name)* с нечувствительным к регистру поиском. Каждая запись в базе представлена структурой *VulnerableFunction*:

```
1 struct VulnerableFunction {  
2     std::string id, name;  
3     std::string category;  
4     std::string algorithm;  
5     std::string quantum_vulnerability;  
6     std::string nist_reference, tc26_reference;  
7     double risk_score = 0.0;  
8     std::vector<std::string> aliases;  
9     std::vector<std::string> patterns;  
10    std::vector<ReplacementInfo> replacements;  
11 };
```

Листинг 28 – Структура записи базы данных уязвимых функций

Механизм псевдонимов позволяет одной записи базы описывать несколько вариантов API: например, устаревший *RSA_generate_key* и актуальный *RSA_generate_key_ex* объединены в одну запись с каноническим именем *RSA_generate_key_ex*. Метод *merge_json* обеспечивает расширяемость: организации могут добавлять собственные записи для корпоративных криптографических обёрток без изменения основной базы.

Тесты. Тест *find_alias* (листинг 29) проверяет ключевое свойство: поиск по псевдониму должен возвращать запись с каноническим именем. Это важно, поскольку анализаторы обнаруживают фактически вызванную функцию, но риск и рекомендации хранятся при каноническом имени.

```
1 TEST(VulnDatabase, find_alias) {
2     pqc::VulnDatabase db;
3     db.load("data/vulnerable_functions.json");
4
5     auto opt = db.find_by_name("RSA_generate_key");
6
7     ASSERT_TRUE(opt.has_value());
8     ASSERT_EQ(opt->name, "RSA_generate_key_ex");
9     ASSERT_EQ(opt->algorithm, "RSA");
10 }
```

Листинг 29 – Тест поиска по псевдониму функции

Тест *replacements_present* проверяет, что поле *standard* в первой рекомендации по миграции содержит ссылку на FIPS — это обязательный атрибут для формирования раздела *replacement* в плане миграции:

```
1 TEST(VulnDatabase, replacements_present) {
2     pqc::VulnDatabase db;
3     db.load("data/vulnerable_functions.json");
4
5     auto opt = db.find_by_name("RSA_generate_key_ex");
6
7     ASSERT_TRUE(opt.has_value());
8     ASSERT_NOT_EMPTY(opt->replacements);
9     ASSERT_CONTAINS(opt->replacements[0].standard, "FIPS");
10 }
```

Листинг 30 – Тест наличия рекомендации по миграции

Полный листинг теста *merge_extra_db*, демонстрирующего добавление корпоративной функции-обёртки, приведён в приложении ??.

5.3.2 Инвентаризация проекта (*ProjectScanner*)

Назначение и структура. Класс *ProjectScanner* рекурсивно обходит директорию проекта и строит объект *ProjectInventory* — центральный носитель метаданных, который передаётся всем анализаторам.

```

1 struct FileEntry {
2     std::filesystem::path path;
3     FileCategory category;
4     std::string language;
5     int line_count = 0;
6 };
7
8 struct ProjectInventory {
9     std::string project_name;
10    std::vector<FileEntry> files;
11    std::map<std::string, int> language_stats;
12
13    std::vector<const FileEntry*> get_source_files() const;
14    nlohmann::json to_cbom_metadata() const;
15 };

```

Листинг 31 – Ключевые структуры инвентаризации проекта

Классификация файлов реализована через плагины *ILanguagePlugin*: встроенные плагины обрабатывают C/C++, CMake и X.509-сертификаты; пользовательский плагин можно зарегистрировать через *register_plugin()*.

Тесты. Тест *cpp_files_detected* проверяет, что после сканирования директории *tests/fixtures* в инвентаре присутствуют файлы *.cpp* в категории *SOURCE* — это предусловие для корректной работы обоих анализаторов:

```

1 TEST(ProjectScanner, cpp_files_detected) {
2     pqc::ProjectScanner scanner;
3     auto inv = scanner.scan("tests/fixtures");
4
5     auto src = inv.get_source_files();
6
7     ASSERT_GT((int)src.size(), 0);
8     bool found_cpp = false;
9     for (auto* f : src)
10         if (f->path.extension() == ".cpp") { found_cpp = true; break; }
11     ASSERT_TRUE(found_cpp);
12 }

```

Листинг 32 – Тест обнаружения C++ файлов при инвентаризации

5.3.3 Regex-анализатор (*RegexAnalyzer*)

Назначение и структура. *RegexAnalyzer* при инициализации компилирует паттерны из базы данных в объекты *std::regex* и хранит их с указателями на соответствующие записи *VulnerableFunction*. При анализе файла анализатор построчно применяет все паттерны к строке, предварительно очищенной от блочных комментариев. Результатом является набор объектов *Finding*.

Структура *Finding* служит универсальным носителем информации об уязвимости и передаётся между всеми последующими модулями конвейера:

```
1 struct Finding {
2     std::string file_path;
3     std::string function_name;
4     std::string library;
5     std::string algorithm;
6     std::string category;
7     std::string quantum_vulnerability;
8     std::string context_function;
9     std::string context_class;
10    std::string nist_reference, tc26_reference;
11    int line_number = 0;
12    double base_risk_score = 0.0;
13    std::vector<std::string> arguments;
14 };
```

Листинг 33 – Структура результата обнаружения уязвимости

Тесты. Тест *no_false_positives_for_safe_code* (листинг 34) является критическим тестом корректности: он проверяет, что симметричный вызов *EVP_EncryptInit_ex* в блоке *SAFE* не порождает находок, поскольку функция отсутствует в базе уязвимых функций (AES-256-GCM — симметричный режим, устойчивый при длине ключа 256 бит даже с учётом алгоритма Гровера).

```
1 TEST(RegexAnalyzer, no_false_positives_for_safe_code) {
2     pqc::VulnDatabase db;
3     db.load("data/vulnerable_functions.json");
4     pqc::RegexAnalyzer ra(db);
5     auto findings = ra.analyze_file(
6         "tests/fixtures/sample_vulnerable.cpp"
7     );
8 }
```

```

8
9     for (auto& f : findings)
10         ASSERT_TRUE(f.function_name != "EVP_EncryptInit_ex");
11 }

```

Листинг 34 – Тест отсутствия ложных срабатываний на симметричный AES-код

Тест *finding_has_nist_reference* перебирает все находки и проверяет, что каждая несёт непустую строку ссылки на стандарт NIST — поле обязательно для раздела *nist_compliance* в JSON-отчёте.

5.3.4 AST-анализатор (*ASTAnalyzer*)

Назначение и структура. *ASTAnalyzer* использует *libclang* для построения дерева разбора исходного файла. Для каждого узла-вызова (*CXCursor_CallExpr*) анализатор:

- 1) определяет имя вызываемой функции через *clang_getCursorReferenced*;
- 2) выполняет поиск в базе *VulnDatabase* по этому имени;
- 3) при обнаружении условно уязвимой функции (*quantum_vulnerability* = *"conditional"*) прослеживает провенанс аргументов через *resolve_expr*, обходя цепочки инициализации переменных назад до литерала или вызова;
- 4) подавляет находку, если аргумент содержит имя постквантового алгоритма (*ml-kem*, *ml-dsa*, *slh-dsa* и др.);
- 5) извлекает контекст: объемлющую функцию, класс и пространство имён через обход родительских курсоров.

При отсутствии *libclang* анализатор автоматически переключается на токенизирующий режим с пониженной точностью извлечения контекста.

Тесты. Тест *context_class_extracted* (листинг 35) проверяет, что AST-анализатор действительно извлекает объемлющий класс. Это важно, поскольку поле *context_class* используется *RiskAssessor* при расчёте контекстных множителей и отображается в отчёте для понимания местоположения уязвимости.

```

1 TEST(ASTAnalyzer, context_class_extracted) {

```

```

2   pqc::VulnDatabase db;
3   db.load("data/vulnerable_functions.json");
4   pqc::ASTAnalyzer aa(db);
5   auto findings = aa.analyze_file(
6       "tests/fixtures/sample_vulnerable.cpp"
7   );
8
9   bool has_class = false;
10  for (auto& f : findings)
11      if (!f.context_class.empty()) { has_class = true; break; }
12  ASSERT_TRUE(has_class);
13 }

```

Листинг 35 – Тест извлечения объемлющего класса AST-анализатором

Тест *no_system_header_findings* проверяет критическое ограничение: все находки должны указывать на файл пользователя, а не на системные заголовки OpenSSL. Без этой проверки анализатор мог бы ошибочно репортировать внутренние объявления библиотечных функций как уязвимые вызовы в проекте.

5.3.5 Оценщик риска (*RiskAssessor*)

Назначение и структура. *RiskAssessor* принимает список объектов *Finding*, вычисляет для каждого итоговую оценку риска и формирует *RiskReport* с отсортированным планом миграции.

Итоговая оценка вычисляется как $\text{final_score} = \min(10, 0, \text{base_score} \times m)$, где мультипликатор m формируется из шести независимых факторов, описанных в структуре *ContextFactors*:

```

1 struct ContextFactors {
2     bool is_network_facing = false;
3     bool is_persistent_data = false;
4     bool is_key_material   = false;
5     bool is_in_loop        = false;
6     bool is_test_code      = false;
7     int  call_frequency    = 1;
8     std::string data_classification;
9 };

```

Листинг 36 – Контекстные факторы и их влияние на оценку риска

На выходе *RiskAssessor::assess* возвращает *RiskReport* — план миграции, отсортированный по убыванию *final_score*, со счётчиками по приоритетам и сводкой по файлам:

```
1 struct MigrationAction {
2     std::string file_path, current_function, current_library;
3     std::string replacement_function, replacement_standard;
4     std::string nist_reference, tc26_note;
5     RiskScore risk;
6     MigrationPriority priority;
7     int migration_order;
8 };
9
10 struct RiskReport {
11     std::vector<MigrationAction> migration_plan;
12     std::map<std::string, FileRiskSummary> file_summaries;
13     double overall_risk_score;
14     std::string nist_readiness;
15 };
```

Листинг 37 – Структуры выходных данных оценщика риска

Тесты. Тест *migration_order_sorted_by_risk* (листинг 38) намеренно передаёт две находки в обратном порядке риска и проверяет, что план миграции упорядочен правильно.

```
1 TEST(RiskAssessor, migration_order_sorted_by_risk) {
2     pqc::VulnDatabase db;
3     db.load("data/vulnerable_functions.json");
4     pqc::RiskAssessor ra(db);
5     pqc::ProjectInventory inv;
6
7     pqc::Finding f1, f2;
8     f1.function_name = "SHA1";
9     f1.base_risk_score = 6.0; f1.file_path = "src/util.cpp";
10    f1.category = "hash_function";
11
12    f2.function_name = "RSA_generate_key_ex";
13    f2.base_risk_score = 10.0; f2.file_path = "src/net.cpp";
14    f2.category = "asymmetric_key_generation";
15
16    auto rr = ra.assess({f1, f2}, inv);
```

```

17
18     ASSERT_EQ((int)rr.migration_plan.size(), 2);
19     ASSERT_GE(rr.migration_plan[0].risk.final_score,
20               rr.migration_plan[1].risk.final_score);
21 }

```

Листинг 38 – Тест сортировки плана миграции по убыванию риска

Тест *test_file_reduces_risk* создаёт находку с путём *tests/test_tls.cpp* и проверяет, что итоговая оценка ниже базовой: *RiskAssessor* распознаёт путь как тестовый файл и применяет коэффициент $\times 0,7$.

5.3.6 Модуль метрик (*MetricsCalculator*)

Назначение и структура. *MetricsCalculator* сопоставляет список объектов *Finding* с эталонной разметкой (*GroundTruth*), используя двойной критерий совпадения: имя функции и номер строки в пределах допуска *line_tolerance*. Результат — объект *AnalysisMetrics* с полями *true_positives*, *false_positives*, *false_negatives* и методами *precision()*, *recall()*, *f1_score()*.

Тест *line_tolerance* (листинг 39) проверяет корректность допуска: находка на строке 45 должна засчитываться как ТР при эталонном значении 42 и допуске 5.

```

1 TEST(MetricsCalculator, line_tolerance) {
2     pqc::MetricsCalculator calc;
3
4     pqc::Finding f;
5     f.function_name = "RSA_sign";
6     f.line_number   = 45;
7
8     pqc::GroundTruth gt;
9     gt.function_name = "RSA_sign";
10    gt.line_number    = 42;
11    gt.line_tolerance = 5;
12
13    auto m = calc.compute({f}, {gt});
14
15    ASSERT_EQ(m.true_positives, 1);
16    ASSERT_EQ(m.false_positives, 0);
17    ASSERT_EQ(m.false_negatives, 0);
18 }

```

Листинг 39 – Тест допуска при сопоставлении строк

5.4 Интеграционные тесты

Интеграционные тесты проверяют полный конвейер обработки на директории *tests/fixtures*. Последовательность вызовов в каждом тесте отражает реальную цепочку исполнения инструмента:

VulnDatabase → *ProjectScanner* → *Analyzer* →
RiskAssessor → *RiskReport*

Тест *regex_pipeline_end_to_end* (листинг 40) последовательно выполняет все этапы и проверяет непустоту плана миграции, ненулевую общую оценку риска и наличие строки *nist_readiness*.

```
1 TEST(FullPipeline, regex_pipeline_end_to_end) {
2     pqc::VulnDatabase db;
3     db.load("data/vulnerable_functions.json");
4
5     pqc::ProjectScanner scanner;
6     auto inv = scanner.scan("tests/fixtures");
7
8     pqc::RegexAnalyzer ra(db);
9     auto findings = ra.analyze(inv);
10    ASSERT_NOT_EMPTY(findings);
11
12    pqc::RiskAssessor assessor(db);
13    auto rr = assessor.assess(findings, inv);
14
15    ASSERT_NOT_EMPTY(rr.migration_plan);
16    ASSERT_GT(rr.overall_risk_score, 0.0);
17    ASSERT_NOT_EMPTY(rr.nist_readiness);
18 }
```

Листинг 40 – Интеграционный тест полного конвейера в режиме regex

Тест *regex_precision_recall_gt50pct* (листинг 41) является регрессионным: он загружает эталонную разметку и проверяет, что все три метрики качества не опускаются ниже 50 %. Порог выбран консервативным, чтобы тест

оставался стабильным при расширении базы, но немедленно сигнализировал о грубом регрессе.

```
1 TEST(FullPipeline, regex_precision_recall_gt50pct) {
2     pqc::VulnDatabase db;
3     db.load("data/vulnerable_functions.json");
4     pqc::ProjectScanner scanner;
5     auto inv = scanner.scan("tests/fixtures");
6     pqc::RegexAnalyzer ra(db);
7     auto findings = ra.analyze(inv);
8
9     auto gt = pqc::MetricsCalculator::load_from_json(
10         "tests/fixtures/ground_truth.json"
11     );
12     pqc::MetricsCalculator calc;
13     auto m = calc.compute(findings, gt);
14
15     ASSERT_GE(m.precision(), 0.5);
16     ASSERT_GE(m.recall(), 0.5);
17     ASSERT_GE(m.f1_score(), 0.5);
18 }
```

Листинг 41 – Интеграционный регрессионный тест метрик качества

Тест *migration_plan_has_fips_references* проверяет сквозное прохождение ссылок на стандарты: поле *replacement_standard* хотя бы одной записи плана миграции должно содержать *"FIPS"* — это подтверждает корректную загрузку и применение рекомендаций из базы. Тест *risk_report_has_tc26_data* аналогично проверяет наличие непустого поля *tc26_note*, что критично для применимости инструмента в проектах с ГОСТ-криптографией.

5.5 Оценка результатов тестирования

Все 47 тестов проходят успешно. Итоги приведены в таблице 5.3.

Таблица 5.3 – Итоги выполнения тестов

Набор тестов	Всего	Пройдено
<i>VulnDatabase</i>	9	9

Продолжение таблицы 5.3

Набор тестов	Всего	Пройдено
<i>ProjectScanner</i>	5	5
<i>RegexAnalyzer</i>	8	8
<i>ASTAnalyzer</i>	7	7
<i>RiskAssessor</i>	7	7
<i>MetricsCalculator</i>	5	5
<i>FullPipeline</i> (интеграционные)	6	6
Итого	47	47

Интеграционный тест *regex_precision_recall_gt50pct* зафиксировал финальные метрики качества на эталонном наборе из 32 записей: для regex-режима — precision = 80,0 %, recall = 100,0 %, F1 = 88,9 %; для AST-режима — precision = 94,1 %, recall = 100,0 %, F1 = 97,0 %. Детальный анализ приведён в разделе 6.2.

6 АНАЛИЗ ПОЛУЧЕННЫХ РЕЗУЛЬТАТОВ

6.1 Соответствие реализации поставленным требованиям

В таблице 6.1 приведено соответствие реализованных компонентов ключевым функциональным требованиям.

Таблица 6.1 – Соответствие реализации функциональным требованиям

Требование	Реализация	Подтверждение
Обнаружение квантово-уязвимых функций в C/C++-коде	Два анализатора: <i>RegexAnalyzer</i> и <i>ASTAnalyzer</i>	ТР = 32/32 в обоих режимах

Продолжение таблицы 6.1

Требование	Реализация	Подтверждение
Поддержка OpenSSL, Crypto++, ГОСТ-криптографии	Три секции в <i>vulnerable_functions.json</i>	Тест <i>gost_library_entries</i>
Условная уязвимость на основе аргументов вызова	<i>ASTAnalyzer: resolve_expr</i> + PQC-фильтр	FP = 0 на <i>safe_pqc_kem</i> в AST-режиме
Расчёт риска с учётом контекста	<i>RiskAssessor: 6 контекстных коэффициентов</i>	Тесты <i>network_file_increases_risk, test_file_reduces_risk</i>
Приоритизированный план миграции	<i>RiskAssessor:: assess: сортировка по final_score</i>	Тест <i>migration_order_sorted_by_risk</i>
Рекомендации с ссылками FIPS и ТК 26	Поля <i>replacement_standard, tc26_note</i> в <i>MigrationAction</i>	Тесты <i>migration_plan_has_fips_references, risk_report_has_tc26_data</i>
CBOM в формате CycloneDX	Класс <i>Cbom</i> , метод <i>ReportGenerator:: build_cbom</i>	Тест <i>report_json_serialization</i>
Расширяемость пользовательской базой	<i>VulnDatabase:: merge_json</i>	Тест <i>merge_extra_db</i>
Измерение precision/recall/F1	<i>MetricsCalculator</i> , флаг <i>--metrics</i>	Тест <i>regex_precision_recall_gt50pct</i>

6.2 Сравнение режимов анализа

Оба режима тестировались на файле *sample_vulnerable.cpp* (32 записи в эталонной разметке). Результаты представлены в таблице 6.2.

Таблица 6.2 – Сравнение метрик качества AST и regex-режимов

Метрика	AST	Regex
Эталонных записей (GT)	32	32
True Positives (TP)	32	32
False Positives (FP)	2	8
False Negatives (FN)	0	0
Precision	94,1 %	80,0 %
Recall	100,0 %	100,0 %
F1	97,0 %	88,9 %
Время анализа тестового файла	≈1030 мс	≈80 мс

True Positives. Оба режима обнаружили все 32 ожидаемые находки. Это охватывает широкий спектр квантово-уязвимых конструкций: прямые вызовы низкоуровневых функций (*RSA_new*, *ECDSA_sign*, *DH_new*), EVP-функции высокого уровня с классическим аргументом (*EVP_PKEY_CTX_new_from_name("RSA",...)*), полные цепочки подписи и верификации через *EVP_DigestSign*/EVP_DigestVerify**, хеш-функции (*SHA1*, *MD5*), а также функции, доступные только через псевдонимы в базе (*EC_KEY_generate_key*, *DH_generate_parameters_ex*).

False Positives. *AST* (2 FP). Обе находки — *BN_new* на строках 50 и 217 — являются фактически корректными срабатываниями: создание *BIGNUM*-структуры служит надёжным индикатором ручной реализации RSA или DH (квантово-уязвимых алгоритмов), но эти строки не вошли в эталонную разметку. Если включить их в GT, precision AST достигает 100 %.

Regex (8 FP). Помимо тех же двух *BN_new* — 6 находок из блока *SAFE*: три вызова в функции *safe_pqc_kem* (*EVP_PKEY_CTX_new_from_name("ML-KEM-768",...)*, *EVP_PKEY_keygen_init*, *EVP_PKEY_keygen*) и аналогичные три вызова из второго PQC-безопасного контекста. Regex-анализатор не мо-

жет отследить провенанс аргумента *"ML-KEM-768"*, поэтому срабатывает на имя функции вне зависимости от переданного алгоритма. AST-анализатор корректно подавляет все эти 6 находок через механизм *resolve_expr*: при обнаружении постквантового имени алгоритма в цепочке аргументов находка не формируется.

False Negatives. В обоих режимах ложноотрицательных результатов нет. Полнота достигается за счёт покрытия базы: она включает как основные имена функций, так и их псевдонимы, а также отдельные записи для хеш-функций и функций генерации параметров, которые иначе могли бы быть пропущены.

Рекомендации по выбору режима. Regex-режим целесообразно использовать как быстрый первичный скрининг: время обработки на тестовом файле — около 80 мс против 1030 мс у AST-режима. AST-режим рекомендуется как основной для формирования итогового отчёта, особенно в проектах, активно использующих EVP-API с передачей алгоритма через строковый аргумент.

6.3 Оценка практической применимости

Помимо поиска уязвимостей, инструмент формирует три класса выходных данных, пригодных для аудита и регуляторной отчётности:

- *SBOM-файл* в формате CycloneDX — стандартизованный инвентарь криптографических активов с полями *primitive* (rsa, ecdh, sha1 и др.), *cryptoFunctions* (keyGeneration, keyEncapsulation, ...) и *quantumSecurityLevel* (0 для полностью уязвимых, 50 для частично ослабленных, 128 для квантово-устойчивых);
- *план миграции* с явными ссылками на FIPS 203/204/205 и рекомендации ТК 26 для переходного периода 2025–2030, включая примеры кода для каждой замены;
- *оценку NIST-готовности* (*not-started* / *in-progress* / *near-compliant* / *compliant*), вычисляемую на основе числа критических и высоких находок.

Время AST-анализа файла объёмом ≈ 2000 строк составляет менее 2 с — это приемлемо для встраивания в CI-конвейер.

6.4 Ограничения реализации

Внутрипроцедурный анализ аргументов. Механизм *resolve_expr* прослеживает провенанс аргументов только в рамках одной функции. Если значение алгоритма передаётся через параметр функции или хранится в поле класса, инициализированном в другом методе, анализатор не сможет определить постквантовую природу аргумента и сформирует ложное срабатывание.

Провенанс аргументов в режиме regex. Regex-анализатор не отслеживает значения переменных, что структурно приводит к 6 неустранимым FP для EVP-функций с постквантовыми аргументами. Любые эвристические правила (белые списки имён переменных) будут хрупкими и не масштабируются на произвольный код.

Зависимость от libclang. AST-анализатор требует установленной libclang и корректно настроенных путей включения. При неполной конфигурации заголовочных файлов часть вызовов может не разрешиться, и анализатор переключается на токенизирующий fallback-режим с пониженной точностью извлечения контекста.

Покрывание базы данных. Текущая база охватывает три библиотеки: OpenSSL, Crypto++ и ГОСТ-криптографию через OpenSSL-провайдер. Функции из wolfSSL, Botan, mbedTLS и других криптографических библиотек не включены и не будут обнаружены.

Отсутствие модуля автоматической миграции. Инструмент предоставляет готовый план миграции с примерами кода, однако не применяет кодовые преобразования автоматически — разработчик выполняет замену вручную на основе сгенерированного отчёта.

6.5 Направления дальнейшего развития

Модуль автоматической миграции кода. Приоритетным направлением является реализация автоматического трансформатора AST, который по записям плана миграции и шаблонам из базы данных генерирует готовые патчи. Например, для нахождения *RSA_generate_key_ex* трансформатор должен сформировать эквивалентный блок с *EVP_PKEY_CTX_new_from_name("ML-DSA-65", ...)*, сохраняя семантику вызова. Технически это требует расширения AST-обходчика до уровня переписывающего трансформера на базе того же cursor-механизма libclang.

Межпроцедурный анализ аргументов. Реализация хотя бы однопроходного межпроцедурного анализа в рамках одной единицы трансляции позволит корректно обрабатывать передачу алгоритма через параметры функций — характерный паттерн в проектах с криптографическими фасадами. Это снизит число как ложноположительных, так и ложноотрицательных результатов на реальном коде.

Расширение базы данных. Добавление функций из wolfSSL, mbedTLS, Botan, а также Python-биндингов (*cryptography*, *pyOpenSSL*), Go-стандартной библиотеки (*crypto/rsa*, *crypto/elliptic*) и Rust-крейтов (*rsa*, *p256*) сделает инструмент применимым для многоязычных проектов.

Интеграция в CI/CD. Добавление режима выхода с ненулевым кодом при обнаружении критических находок (*--fail-on-critical*) и формирование SARIF-отчёта позволят встраивать инструмент в GitHub Actions, GitLab CI и аналогичные системы в качестве обязательного шага перед слиянием.

Временно-зависимая оценка риска. Текущий оценщик риска не учитывает временной горизонт появления криптографически значимых квантовых компьютеров. Добавление временно-зависимой компоненты — например, повышающего коэффициента для долгосрочных ключей с учётом прогноза к 2030 г. — сделает приоритизацию плана миграции более реалистичной и практически значимой.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *IBM Institute for Business Value, Cloud Security Alliance*. Secure the post-quantum future: Global readiness, risks, and opportunities : Industry report / IBM Institute for Business Value ; Cloud Security Alliance. — 2025. — URL: <https://www.ibm.com/downloads/documents/us-en/1443d5f5af4f5316> ; IBM Quantum Safe / IBM Institute for Business Value report.
2. *Gidney C.* How to factor 2048 bit RSA integers with less than a million noisy qubits. — 2025. — eprint: arXiv:2505.15917. — URL: <https://arxiv.org/abs/2505.15917> (дата обр. 10.05.2026).
3. *Mosca M., Piani M.* Quantum Threat Timeline Report 2025 : тех. отч. / Global Risk Institute ; evolutionQ Inc. — 2026. — URL: <https://globalriskinstitute.org/publication/quantum-threat-timeline-report-2025b/> (дата обр. 10.05.2026).
4. *Ключарев П. Г.* Основы квантовых вычислений и квантовой криптографии // International Journal of Open Information Technologies. — 2022. — URL: <https://cyberleninka.ru/article/n/osnovy-kvantovyh-vychisleniy-i-kvantovoy-kriptografii> (дата обр. 10.05.2026).
5. Квантовые алгоритмы: алгоритм Шора и алгоритм Гровера // Инновационная наука. — 2025. — URL: <https://cyberleninka.ru/article/n/kvantovye-algoritmy-algorithm-shora-i-algorithm-grovera> (дата обр. 10.05.2026).
6. Квантовый компьютер и криптографическая стойкость современных систем шифрования // Материалы российской конференции по информационной безопасности. — 2023. — URL: <https://cyberleninka.ru/article/n/kvantovyy-kompyuter-i-kriptograficheskaya-stoykost-sovremennyh-sistem-shifrovaniya> (дата обр. 10.05.2026).
7. Implementing Grover's on AES-based AEAD schemes / A. Saha [и др.] // Scientific Reports. — 2024. — Т. 14. — С. 21105. — DOI: 10.1038/s41598-024-69188-8.

8. *National Institute of Standards and Technology*. NIST IR 8547 (Initial Public Draft). Transition to Post-Quantum Cryptography Standards : тех. отч. / U.S. Department of Commerce. — 2024. — URL: <https://csrc.nist.gov/pubs/ir/8547/ipd> (дата обр. 10.05.2026).
9. *National Institute of Standards and Technology*. FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. — 2024. — DOI: 10.6028/NIST.FIPS.203. — URL: <https://csrc.nist.gov/pubs/fips/203/final> (дата обр. 10.05.2026).
10. *Кустов Е. Ф., Беззатеев С. В.* Анализ применимости существующих схем разделения секрета в условиях постквантовой эры // Научно-технический вестник информационных технологий, механики и оптики. — 2025. — URL: <https://ntv.elpub.ru/jour/article/view/468> (дата обр. 10.05.2026).